

**KOCAELİ ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ**

**ELEKTRONİK VE HABERLEŞME MÜHENDİSLİĞİ
BÖLÜMÜ**

BİTİRME ÇALIŞMASI

**GÖMÜLÜ SİSTEM ÜZERİNDE YAPAY ZEKA
DESTEKLİ DOST-DÜŞMAN HEDEF TESPİT VE TAKİP
SİSTEMİ**

SUDE ÇAKIR

DANIŞMAN: Doç. Dr. Aysun Taşyapı ÇELEBİ

KOCAELİ, 2026

ÖNSÖZ ve TEŞEKKÜR

Bu çalışma, günümüzde hava sahası güvenliğini tehdit eden unsurların otonom sistemlerle ayırt edilmesi ve izlenmesi amacıyla yürütülen "**Gömülü Sistemlerde Derin Öğrenme Tabanlı Drone ve Kuş Tespit ve Takip Sistemi**" projesinin aşamalarını kapsamaktadır. Proje kapsamında, kısıtlı donanım kaynaklarına sahip gömülü platformlarda yüksek doğrulukla nesne tespiti yapılması ve askeri standartlarda bir takip arayüzü sunulması hedeflenmiştir.

Lisans eğitimim boyunca edindiğim teorik bilgileri bu çalışma ile pratiğe dökme şansı bulurken, projenin her aşamasında değerli vaktini ayırarak tecrübelerini paylaşan, teknik vizyonu ile çalışmamı ışık tutan kıymetli danışman hocam **Doç. Dr. Aysun Taşyapı ÇELEBİ**'ye sonsuz teşekkürlerimi sunarım. Hocamın yapıcı eleştirileri ve akademik rehberliği, bu çalışmanın olgunlaşmasında temel rol oynamıştır.

Çalışma sürecinde teknik imkanlarından yararlandığım **Kocaeli Üniversitesi Elektronik ve Haberleşme Mühendisliği** bölümündeki değerli hocalarıma ve akademik gelişimime katkı sağlayan tüm çalışma arkadaşlarıma şükranlarımı sunarım.

Son olarak, bu uzun soluklu eğitim sürecinde sabırlarını ve desteklerini hiçbir zaman esirgemeyen, motivasyonumun en büyük kaynağı olan sevgili aileme en içten teşekkürlerimi iletirim.

Haziran 2026, KOCAELİ

Sude ÇAKIR

İÇİNDEKİLER

ÖNSÖZ ve TEŞEKKÜR.....	i
İÇİNDEKİLER	ii
ŞEKİLLER DİZİNİ.....	iv
TABLolar DİZİNİ	v
SİMGELER DİZİNİ VE KISALTMALAR	vi
Gömülü sistem üzerinde YAPAY ZEKA DESTEKLİ dost-düşman hedef tespit ve takip sistemi	vii
ai-powered friend or foe target detection and tracking system on embedded system.....	viii
1. GİRİŞ	9
1.1 PROBLEM TANIMI VE HEDEFLER	11
1.1.1 PROBLEM TANIMI	11
1.1.2 HEDEFLER	11
2. KULLANILAN ARAÇLAR VE TEKNOLOJİLER	12
2.1 Raspberry Pi 5.....	12
2.2 Raspberry Pi Camera 3 Wide	13
2.3 You Only Look Once (YOLO).....	14
2.3.1 YOLO Genel Mimarisi	16
2.3.2 YOLOv8	17
2.3.3 YOLO26	18
2.3.4 YOLOv8n ve YOLO26n Karşılaştırması	19
2.4 Neural Network Computing Framework (NCNN)	20
2.5 Açık Kaynak Bilgisayarlı Görü Kütüphanesi (OpenCV)	21
2.6 Python Programlama Dili	22
2.7 Roboflow	22
2.8 Google Colab.....	23
2.9 PCA9685 Servo Sürücü Kartı.....	24
2.10 Pan-Tilt Mekanizması.....	25
3. sistem tasarımı ve oluşturulması	26
3.1 Sistemin Genel Çalışma Mantığı ve Akış Diyagramı	26
3.1.1 Görüntü Giriş Katmanı ve Ön İşleme	27
3.1.2 Yapay Zeka ve Karar Motoru (Merkezi İşlem)	27
3.1.3 Sistem Çıktıları ve Mekanik Kontrol.....	28
3.2 Veri Seti Hazırlama ve Etiketleme Süreci	28
3.2.1 Veri Kaynakları ve Ham Veri Toplama.....	28
3.2.2 Video Kayıtlarından Manuel Kare Çıkarımı.....	29
3.2.3 Veri Seti Genel Özellikleri.....	31
3.2.4 Veri Etiketleme Süreci (Roboflow)	33
3.2.5 Zorlayıcı arka planlar ve etiketsiz (negatif) örnekler.....	34
3.2.6 Veri setinin sürüm bazlı geliştirilmesi	34
4. MODELLEME VE DENEYSEL ÇALIŞMALAR	36
4.1 Problem biçimlendirmesi.....	36
4.2 Yolo Modelleri ve Eğitim ve Süreci.....	37

4.2.1	Eğitim Ortamı ve Hiperparametreler	37
4.2.2	Veri Seti ile Optimizasyon ve İteratif Başarım Artışı.....	38
4.2.3	YOLOv8n ve YOLO26n Sonuçları Üzerinden Mimari Karşılaştırma	39
4.3	Hedef Tespiti ve Kararlı Takip Yaklaşımı.....	41
4.3.1	Küçük Nesnelerde Tespit Problemi	41
4.3.2	Kalman Filtresi ile YOLO Entegrasyonu	43
4.3.3	Optik Akış (Optical Flow) ile YOLO Entegrasyonu	45
4.3.4	İlgi Alanı (ROI) Tabanlı Yeniden Tespit Yaklaşımı	46
4.3.5	Zamansal Oy Çoğunluğu ile Kararlı Hedef Sınıflandırması.....	49
4.3.6	Çoklu Hedef Takibi, ID Ataması ve HUD Arayüzü.....	51
4.4	Raspberry Pi 5 Gömülü Platform Üzerinde Sistemin İlk Uygulaması	52
4.4.1	İşletim Sisteminin Kurulumu	53
4.4.2	YOLO Modelinin PyTorch Formatında Testi.....	54
4.4.3	YOLO Modelinin NCNN Formatına Dönüşümü ve Testi.....	55
4.4.4	YOLOv8n Modelinin Farklı Çözünürlükteki Görüntü Kaynakları ile Karşılaştırılması	57
4.5	Nihai Takip Sisteminin Entegrasyonu ve Performans Değerlendirmesi	59
5.	PAN-TİİT DESTEKLİ GERÇEK ZAMANLI OTONOM TAKİP MEKANİZMASININ UYGULANMASI	62
5.1	Otonom Takip Sisteminin Donanım Mimarisi ve İletişim Protokolleri	63
5.2	Pan-Tilt Destekli Otonom Takip Algoritmasının Tasarımı	64
5.3	Nihai Sonuç Mimarisi.....	66
5.4	Açık Alan Saha Testleri ve Deneysel Sonuçlar	68
	SONUÇLAR ve ÖNERİLER.....	71
	KAYNAKLAR	73
	ÖZGEÇMİŞ	74

ŞEKİLLER DİZİNİ

Şekil 1.1: Görüntü işleme, tespit–sınıflandırma, takip ve kontrol birimlerini gösteren genel sistem mimarisi	10
Şekil 2.1: Raspberry Pi 5	12
Şekil 2.2: Raspberry Pi Camera 3 Wide	14
Şekil 2.3: Tek aşamalı (YOLO) ve iki aşamalı (R-CNN) dedektör çalışma prensiplerinin karşılaştırması	15
Şekil 2.4: Genel YOLO Mimarisi	16
Şekil 2.5: Backbone, Neck ve Head Bileşenlerinin Operasyonel Yapısı	17
Şekil 2.6: Derin Öğrenme Modeli Dönüştürme Süreci	21
Şekil 2.7: OpenCV ve Python kullanımı	21
Şekil 2.8: Veri hazırlama, model eğitimi ve model çıktısı elde etme süreci	23
Şekil 2.9: PCA9685 Servo Sürücü Kartı	24
Şekil 2.10: Pan-Tilt Mekanizması	25
Şekil 3.1: Dost-Düşman Hedef Tespit ve Otonom Takip Diyagramı	26
Şekil 3.2: Örnek Veri Görüntüleri	29
Şekil 3.3: Video kayıtlarından kare çıkarma sürecinin akış şeması	30
Şekil 3.4: Veri setinin Roboflow platformu üzerindeki görünümü	31
Şekil 3.5: Veri setinde kullanılan sınıflar (drone ve kuş)	32
Şekil 3.6: Veri setinin eğitim, doğrulama ve test kümelerine dağılımı	32
Şekil 3.7: Veri setine uygulanan ön işleme ve veri artırma ayarları	33
Şekil 3.8: Etiketlenmiş Drone ve Kuş Görüntüleri	33
Şekil 3.9: Etiketsiz Görüntüler	34
Şekil 3.10: Veri setinin versiyonlama ve güncelleme süreci	35
Şekil 4.1: İki sınıflı tespit örneği: kuş ve drone için sınır kutuları	36
Şekil 4.2: Tespit Sınırlarının Kaybolması ve Kararsız Tespitler	42
Şekil 4.3: Kalman Filtresinin Yapısı [8]	43
Şekil 4.4: YOLO+Kalman Filtresi Yaklaşımının Çıktıları	44
Şekil 4.5: Optical Flow Akış Diyagramı [6]	45
Şekil 4.6: YOLO+Kalman+Optical Flow Yaklaşımının Çıktıları	46
Şekil 4.7: Kullanılan ROI Yapısını Gösteren Diyagram	47
Şekil 4.8: YOLO+ROI Yaklaşımının Çıktıları	48
Şekil 4.9: Kararsız Tespitlerin Örnek Çıktıları	50
Şekil 4.10: Doğru Kararlı Tespitlerin Örnek Çıktıları	50
Şekil 4.11: Düşman(Drone) Hedefinin Hud Arayüzü ile İşaretlenmesi 1	52
Şekil 4.12: Düşman(Drone) Hedefinin Hud Arayüzü ile İşaretlenmesi 2	52
Şekil 4.13: Pytorch Modelin Performansının Terminal Çıktısı	55
Şekil 4.14: YOLOv8n modelinin PyTorch formatından NCNN formatına dönüştürülmesi ..	56
Şekil 4.15: NCNN Modelin Performansının Terminal Çıktısı	57
Şekil 4.16: Düşük Çözünürlükü Kaynağın Performansının Terminal Çıktısı	58
Şekil 4.17: Yüksek Çözünürlükü Kaynağın Performansının Terminal Çıktısı	58
Şekil 4.18: Sistemin Akış Diyagramı	60
Şekil 4.19: Test Sonuçları	61
Şekil 5.1: Pan-Tilt Destekli Otonom Hedef Takip Sisteminin Donanım Mimarisi	64
Şekil 5.2 : Sapma (Error) Hesaplama Mimarisi	65
Şekil 5.3: Gerçek Zamanlı Arayüzdeki Hesaplanan Bilgiler	66
Şekil 5.4: Nihai Sistem Mimarisi	67
Şekil 5.5: Projenin Fiziksel Hali	68
Şekil 5.6: Gerçek Zamanlı Test Sonuçları	69
Şekil 5.7 : Gerçek Zamanlı Test Görüntüsü	70

TABLÖLAR DİZİNİ

Tablo 2.1: Raspberry Pi 5 Teknik Özellikler.....	13
Tablo 2.2: Raspberry Pi Camera 3 Wide Teknik Özellikler	14
Tablo 2.3: YOLOv8n ve YOLO26n Modellerinin Karşılaştırılması	19
Tablo 4.1: Modellerin Başarım Metrikleri.....	39
Tablo 4.2: YOLOv8n ve YOLO26n Karşılaştırması	40
Tablo 4.3: Kalman, Optical Flow ve ROI Yaklaşımlarının Karşılaştırılması.....	48
Tablo 4.4: Raspberry Pi 5 için Uygun İşletim Sistemleri ve Özellikleri.....	53

SİMGELER DİZİNİ VE KISALTMALAR

AI:	Yapay Zeka (Artificial Intelligence)
ARM:	Gelişmiş RISC Makinesi (Advanced RISC Machine)
CNN:	Evrişimsel Sinir Ağları (Convolutional Neural Networks)
COCO:	Ortak Nesneler Bağlam İçinde (Common Objects in Context)
CPU:	Merkezi İşlem Birimi (Central Processing Unit)
DSI:	Ekran Seri Arayüzü (Display Serial Interface)
FLOPs:	Saniyedeki Kayan Nokta İşlemleri (Floating Point Operations per Second)
FOV:	Görüş Alanı (Field of View)
FPS:	Saniye Başına Kare Sayısı (Frames Per Second)
GPIO:	Genel Amaçlı Giriş/Çıkış (General Purpose Input/Output)
GPU:	Grafik İşlem Birimi (Graphics Processing Unit)
HDR:	Yüksek Dinamik Aralık (High Dynamic Range)
HUD:	Baş Üstü Göstergesi (Heads-Up Display)
I2C:	Cihazlar Arası İletişim (Inter-Integrated Circuit)
ID:	Kimlik Numarası (Identification)
İHA:	İnsansız Hava Aracı
IoU:	Kesişim Bölü Birleşim (Intersection over Union)
LCK:	Kilitli Hedef (Locked)
mAP:	Ortalama Hassasiyet Değeri (Mean Average Precision)
MOT:	Çoklu Nesne Takibi (Multi-Object Tracking)
NCNN:	Mobil Platformlar İçin Optimize Edilmiş Sinir Ağı Kütüphanesi
NMS:	Maksimum Olmayanı Bastırma (Non-Maximum Suppression)
ONNX:	Açık Sinir Ağı Değişimi (Open Neural Network Exchange)
OpenCV:	Açık Kaynak Bilgisayarlı Görü Kütüphanesi
PWM:	Sinyal Genişlik Modülasyonu (Pulse Width Modulation)
RAM:	Rastgele Erişimli Bellek (Random Access Memory)
ROI:	İlgi Alanı (Region of Interest)
SIMD:	Tek Komut, Çoklu Veri (Single Instruction, Multiple Data)
TRK:	Takip Edilen Nesne (Tracked)
VNC:	Sanal Ağ Hesaplaması (Virtual Network Computing)
YOLO:	Sadece Bir Kez Bak (You Only Look Once)

GÖMÜLÜ SİSTEM ÜZERİNDE YAPAY ZEKA DESTEKLİ DOST-DÜŞMAN HEDEF TESPİT VE TAKİP SİSTEMİ

SUDE ÇAKIR

Anahtar Kelimeler: Derin Öğrenme, Gömülü Sistemler, Nesne Tespiti ve Takibi, Dost-Düşman Tanıma, Gerçek Zamanlı Görüntü İşleme

Özet: Günümüzde hava sahasında güvenliği tehdit edebilen insansız hava araçlarının (İHA) erken aşamada tespit edilmesi ve doğru biçimde sınıflandırılması, hem savunma uygulamaları hem de sivil güvenlik açısından büyük önem taşımaktadır. Bu çalışma kapsamında, Raspberry Pi 5 tabanlı bir gömülü platform üzerinde gerçek zamanlı çalışabilen, dost-düşman hedef tespiti ve takibine yönelik görüntü işleme destekli bir sistem geliştirilmiştir.

Önerilen yapıda, YOLO tabanlı derin öğrenme modeli gömülü donanımda verimli çalışabilmesi için NCNN formatına dönüştürülmüş; kesintisiz hedef takibi amacıyla Bölge Odaklı Arama (Region of Interest, ROI) yaklaşımı kullanılmış ve operasyonel kullanım senaryolarını destekleyen Heads-Up Display (HUD) tabanlı bir arayüz tasarlanmıştır. Geliştirilen sistem, görüntüde tespit edilen hedefleri düşman (drone) ve dost (kuş) olarak ayırt etmekte; en yakın tehdit olarak değerlendirilen hedefe kilitlenerek takip işlemini sürdürmekte ve tüm bu adımları sınırlı donanım kaynaklarıyla gerçek zamanlı olarak gerçekleştirebilmektedir.

AI-POWERED FRIEND OR FOE TARGET DETECTION AND TRACKING SYSTEM ON EMBEDDED SYSTEM

SUDE ÇAKIR

Keywords: Deep Learning, Embedded Systems, Object Detection and Tracking, Friend–Foe Identification, Real-Time Image Processing

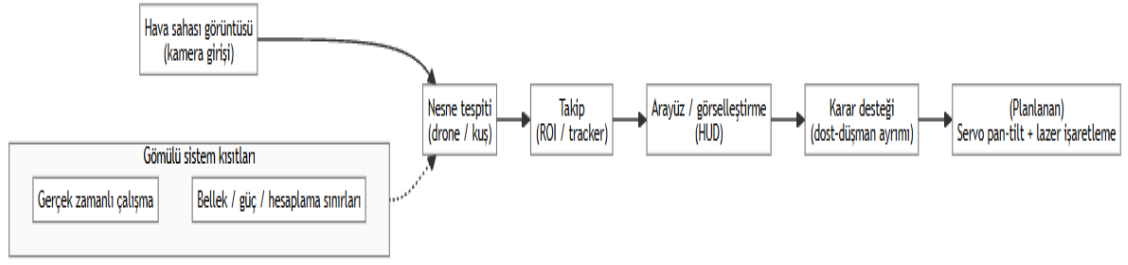
Abstract: With the increasing use of unmanned aerial vehicles (UAVs), detecting and correctly identifying potential threats in the airspace has become a critical need for both defense and civilian security applications. In this study, an AI-assisted image-processing system is developed on a Raspberry Pi 5–based embedded platform to enable real-time friend–foe target detection and tracking.

To run a YOLO-based deep learning model efficiently on the embedded device, the model is converted to the NCNN format. Continuous target tracking is supported using a Region of Interest (ROI) approach, and a Heads-Up Display (HUD) interface is designed to present the system output in an operationally suitable manner. The proposed system classifies detected objects as foe (drone) and friend (bird), locks onto the nearest threat for tracking, and performs all processing steps in real time under limited hardware resources.

1. GİRİŞ

İnsansız hava araçlarının (İHA) sivil ve askeri alanlarda yaygınlaşmasıyla birlikte, hava sahasının izlenmesi ve yetkisiz unsurların erken aşamada tespit edilmesi kritik bir ihtiyaç hâline gelmiştir. Özellikle küçük boyutlu drone'lar; düşük maliyetleri, kolay erişilebilir olmaları ve farklı amaçlarla kullanılabilmeleri nedeniyle güvenlik açısından yeni riskler doğurmaktadır. Bununla birlikte, kuşlar gibi doğal hedefler de görüntü tabanlı sistemlerde drone'lara benzer görsel özellikler sergileyebildiğinden yanlış alarm üretimine neden olabilmekte; bu durum hem operatör yükünü artırmakta hem de karar süreçlerinin güvenilirliğini azaltabilmektedir. Dolayısıyla, drone ve kuş gibi hedeflerin doğru biçimde ayrıştırılması, gerçek ortamda çalışacak bir sistem için temel gereksinimlerden biridir.

Son yıllarda derin öğrenme tabanlı görüntü işleme yöntemleri, gerçek zamanlı nesne tespitinde yüksek başarımlar sağlamış; ancak bu yöntemlerin gömülü sistemler üzerinde çalıştırılması hesaplama gücü, bellek kapasitesi ve enerji tüketimi gibi kısıtlar nedeniyle önemli mühendislik zorluklarını beraberinde getirmiştir. Uygulama tarafında yalnızca yüksek doğruluk yeterli olmayıp, sistemin düşük gecikmeyle kesintisiz çalışması ve çıktılarının sahada anlaşılır/operasyonel bir biçimde sunulması da gerekmektedir. Bu bağlamda nesne tespit çıktılarının takip (tracking) mekanizmalarıyla desteklenmesi; hedefin kareler arasında sürekliliğinin korunması, anlık kayıpların telafi edilebilmesi ve sistemin daha kararlı çalışması açısından önem taşır [9]. Bu kapsamda, sistemin genel çalışma akışı ve bileşenler arası etkileşim, Şekil 1.1'de şematik olarak gösterilmektedir.



Şekil 1.1: Görüntü işleme, tespit–sınıflandırma, takip ve kontrol birimlerini gösteren genel sistem mimarisi.

Bu tez çalışması, kısıtlı donanım kaynaklarına sahip gömülü bir platform üzerinde gerçek zamanlı çalışabilecek bir nesne tespit yaklaşımı ile drone ve kuş hedeflerinin tespit edilmesi, sınıflandırılması ve takip edilmesini amaçlamaktadır. Çalışma kapsamında proje gereksinimlerine uygun bir veri seti hazırlanmış, Roboflow üzerinde sürümleme yaklaşımıyla veri kapsamı iteratif biçimde genişletilmiştir [7]. YOLO tabanlı farklı model mimarileri ile eğitim deneyleri yürütülmüş; elde edilen doğrulama sonuçları precision, recall, mAP50 ve mAP50-95 gibi ölçütlerle değerlendirilerek hız–başarım dengesi gözetilmiştir [2]. Ayrıca, tespit ve takip çıktılarının kullanıcıya anlaşılır biçimde sunulabilmesi için bir izleme/görselleştirme arayüzü yaklaşımı ele alınmış; çalışmanın devamında tespit edilen hedefin konum bilgisinden yararlanarak servo tabanlı yönlendirme ve lazer ile işaretleme mekanizmasının sisteme entegre edilmesi planlanmıştır.

Bu kapsamda tezde; problem tanımı ve motivasyon, veri setinin hazırlanması ve etiketleme süreci, kullanılan yöntem ve modelleme yaklaşımı, eğitim–değerlendirme sonuçları ve sistemin genel mimarisi ayrıntılı olarak sunulmaktadır. Çalışmanın nihai hedefi, hava sahasında gözlemlenen hedeflerin güvenilir biçimde tespit edilmesi ve izlenmesi için gömülü sistem üzerinde çalışabilen, gerçek zamanlı ve uygulanabilir bir çözüm ortaya koymaktır.

1.1 PROBLEM TANIMI VE HEDEFLER

1.1.1 PROBLEM TANIMI

- Hava sahasında görülen küçük hedeflerin (özellikle drone) erken tespiti güvenlik açısından kritiktir.
- Görüntü tabanlı sistemlerde kuşlar, drone'larla benzer boyut/siluet/hareket özellikleri gösterebildiği için yanlış alarm üretir.
- Derin öğrenme ile tespit mümkün olsa da, çözümün gömülü sistemde gerçek zamanlı çalışması; işlem gücü, bellek ve enerji kısıtları nedeniyle zordur.
- Bu nedenle hem doğru sınıflandırma (drone–kuş ayrımı) hem de kararlı takip (hedefin ekranda sürekliliği) aynı anda sağlanmalıdır.

1.1.2 HEDEFLER

- Gömülü platform üzerinde gerçek zamanlı çalışabilecek tespit–sınıflandırma–takip sistemi geliştirmek.
- Hedefleri drone/kuş olarak ayırt ederek yanlış alarmları azaltmak ve tehdit ayrımı için karar desteği sağlamak.
- Tespit edilen hedefin konum bilgisini kullanarak servo motor tabanlı pan-tilt yönlendirme ile hedefi takip etmek.
- Takip edilen hedefi kullanıcıya göstermek ve doğrulamak amacıyla lazer ile işaretleme mekanizması tasarlamak/uygulamak.
- Uçtan uca sistemi doğruluk, gecikme (latency), FPS ve takip kararlılığı gibi metriklerle değerlendirmek.

2. KULLANILAN ARAÇLAR VE TEKNOLOJİLER

2.1 Raspberry Pi 5

Raspberry Pi 5, Raspberry Pi Foundation tarafından geliştirilen ve tek kart üzerinde işlemci, bellek, giriş-çıkış birimleri ve bağlantı arayüzlerini bir arada sunan bir tek kartlı bilgisayar (Single Board Computer, SBC) örneğidir. Düşük güç tüketimi, kompakt boyut ve göreceli olarak yüksek işlem kapasitesi sayesinde prototip geliştirme, eğitim uygulamaları ve gömülü sistem projelerinde yaygın biçimde tercih edilmektedir [3]. Bu çalışmada da sistemin sahada taşınabilir ve sürekli çalışabilir bir donanım üzerinde yürütülebilmesi hedefi doğrultusunda Raspberry Pi 5 gömülü platform olarak seçilmiştir. Bu kapsamda, çalışmada kullanılan Raspberry Pi 5 platformunun fiziksel görünümü Şekil 2.1’de verilmiştir.



Şekil 2.1: Raspberry Pi 5

Raspberry Pi 5, önceki nesillere kıyasla daha güçlü bir işlemci mimarisi ve geliştirilmiş grafik/işlem alt sistemi sunarak özellikle gerçek zamanlı görüntü işleme ve hafif/optimize edilmiş derin öğrenme çıkarımı gibi iş yüklerinde daha iyi bir denge sağlayabilmektedir. Bu özellik, kamera görüntüsünün sürekli akış halinde işlenmesi ve tespit sonuçlarının eşzamanlı olarak arayüze aktarılması gereken uygulamalar için önemlidir. Ayrıca cihazın sunduğu USB, Ethernet, GPIO ve kamera bağlantısı gibi arayüzler; görüntü yakalama, iletişim ve ilerleyen aşamalarda planlanan servo tabanlı yönlendirme entegrasyonu açısından esnek bir donanım zemini oluşturur.

Bu tez kapsamında Raspberry Pi 5, YOLO tabanlı modelin NCNN gibi hafif çıkarım çerçeveleriyle gömülü ortamda çalıştırılması, ROI tabanlı takip mantığının

uygulanması ve HUD arayüzünün işletilmesi için merkezi işlem birimi olarak konumlandırılmıştır. Böylece sistem, yüksek güçlü sunucu donanımına ihtiyaç duymadan sahada denenebilir ve geliştirilebilir bir yapı kazanmıştır. Çalışmada kullanılan Raspberry Pi 5 gömülü platformunun teknik donanım özellikleri Tablo 2.1’de verilmiştir.

Tablo 2.1: Raspberry Pi 5 Teknik Özellikler

Özellik	Açıklama
İşlemci (CPU)	2.4 GHz, 4 çekirdekli ARM Cortex-A76 (BCM2712)
Grafik (GPU)	VideoCore VII
RAM	4 GB / 8 GB LPDDR4X-4267
USB portları	2 × USB 3.0, 2 × USB 2.0
Video çıkışı	2 × micro HDMI (4Kp60 desteği)
Ethernet	Gigabit Ethernet
Kablosuz bağlantı	Çift bant IEEE 802.11ac; Bluetooth 5.0 / BLE
GPIO	40 pin başlık
Depolama	microSD kart
Güç girişi	USB-C (5 V, üretici önerisine uygun güç kaynağı)
PCIe	PCIe 2.0 ×1 (FPC bağlantısı)

2.2 Raspberry Pi Camera 3 Wide

Görüntü işleme ve nesne tespiti süreçlerinde veri girişinin kalitesi, sistemin doğruluğunu doğrudan etkilemektedir. Bu projede, geniş alan taraması yapabilmek ve hareketli hedeflerin (İHA ve kuşlar) takibini kolaylaştırmak amacıyla Raspberry Pi Camera Module 3 Wide tercih edilmiştir. Geniş açılı görüntüleme yeteneğine sahip Raspberry Pi Camera Module 3 Wide donanımının fiziksel yapısı Şekil 2.2’de gösterilmektedir.



Şekil 2.2: Raspberry Pi Camera 3 Wide

Sony IMX708 sensörü üzerine inşa edilen bu modül, 11.9 megapiksel çözünürlüğü ve HDR (High Dynamic Range) desteği sayesinde değişken ışık koşullarında dahi net görüntüler elde edilmesine olanak tanır [3]. Standart kamera modüllerinden farklı olarak 120 derecelik geniş görüş açısı (diagonal FOV) sunması, İHA tespiti gibi geniş bir hava sahasının izlenmesi gereken senaryolarda kör noktaları minimize etmektedir. Çalışmada kullanılan Raspberry Pi Camera Module 3 Wide görüntüleme biriminin teknik özellikleri Tablo 2.2’de verilmiştir.

Tablo 2.2: Raspberry Pi Camera 3 Wide Teknik Özellikler

Özellik	Açıklama
Sensör	Sony IMX708
Çözünürlük	11.9 Megapiksel (4608×2592 piksel)
Görüş Açısı (Diagonal)	120 Derece (Geniş Açılı)
Görüş Açısı (Yatay/Dikey)	102° Yatay / 67° Dikey
Odaklama (Focus)	Faz Algılamalı Otomatik Odaklama (PDAF)
Odak Aralığı	5 cm – ∞
Diyafram (Aperture)	F2.2
Video Modları	1080p50, 720p100, 480p120
HDR Desteği	Var (3 MP çözünürlüğe kadar)
Yazılım Uyumluluğu	libcamera, Picamera2

2.3 You Only Look Once (YOLO)

You Only Look Once" (YOLO), geleneksel nesne tespit sistemlerinden temel bir felsefe farkıyla ayrılmaktadır. Bu model, nesne tespitini bir sınıflandırma problemi

yerine, doğrudan görüntü piksellerinden sınıf olasılıklarına ve sınırlayıcı kutu (bounding box) koordinatlarına giden tek bir regresyon problemi olarak ele almaktadır [4].

YOLO'nun temel çalışma mantığı, girdi görüntüsünü $S \times S$ boyutlarında eşit karelere bölünmüş bir ızgara (grid) yapısına ayırmaktır [4]. Eğer bir nesnenin merkezi belirli bir hücrenin içine düşerse, o hücre söz konusu nesneyi tespit etmekten sorumlu olur. Her hücre; nesnenin konumu, genişliği, yüksekliği ve o bölgede bir nesne bulunma olasılığını temsil eden bir güven skoru (confidence score) tahmin eder. Bu uçtan uca (end-to-end) eğitim ve tahmin süreci, sinir ağının görüntüdeki nesneleri ve bu nesnelerin birbirleriyle olan ilişkilerini bütünsel bir perspektifle öğrenmesini sağlar.

YOLO'nun literatürdeki en büyük başarısı, "doğrulukta (accuracy) makul bir miktar ödün vererek, işlem hızında (inference speed) devrim yaratmasıdır". Bu özellik, özellikle drone (İHA) tespiti ve takibi gibi ani gelişen olaylara saniyeler içinde tepki verilmesi gereken savunma ve güvenlik projelerinde YOLO'yu vazgeçilmez kılmaktadır. YOLO tabanlı tek aşamalı yaklaşım ile R-CNN tabanlı iki aşamalı yaklaşımın işlem akışları ve mimari farklılıkları Şekil 2.3'te karşılaştırmalı olarak sunulmaktadır.

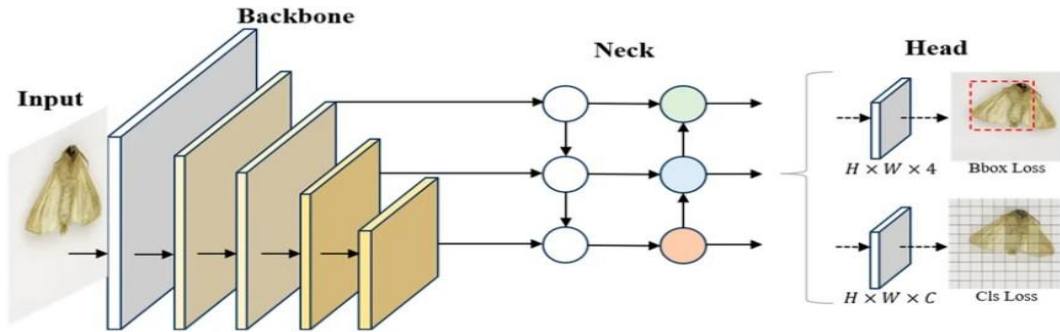


Şekil 2.3: Tek aşamalı (YOLO) ve iki aşamalı (R-CNN) dedektör çalışma prensiplerinin karşılaştırması

- **İki Aşamalı Sistemlerde:** Diyagramda görüldüğü üzere süreç, nesne olabilecek binlerce bölgenin (Region Proposals) tespitiyle başlar. Bu bölgeler tek tek kırılarak sınıflandırıcıya gönderilir. Bu durum, aynı görüntünün farklı bölümleri için sinir ağının defalarca çalışmasına neden olur ve büyük bir hesaplama yükü (gecikme) yaratır.
- **YOLO (Tek Aşamalı) Sisteminde:** Şekil 2.3'ün alt kısmında görüldüğü gibi, görüntü ağı bir kez girer ve doğrudan sonuç üretilir. Ara bir "bölge öneri" katmanını bulunmadığı için gecikme süreleri milisaniyeler seviyesine iner.

2.3.1 YOLO Genel Mimarisi

Modern YOLO modellerinin modüler yapısı, sistemin hem farklı donanımlara (örneğin Raspberry Pi 5) kolayca uyarlanmasını hem de farklı ölçeklerdeki nesneleri etkin biçimde tespit edebilmesini sağlar. Modern YOLO mimarisinin temel bileşenleri ve veri akışı Şekil 2.4'de görsel olarak sunulmaktadır.

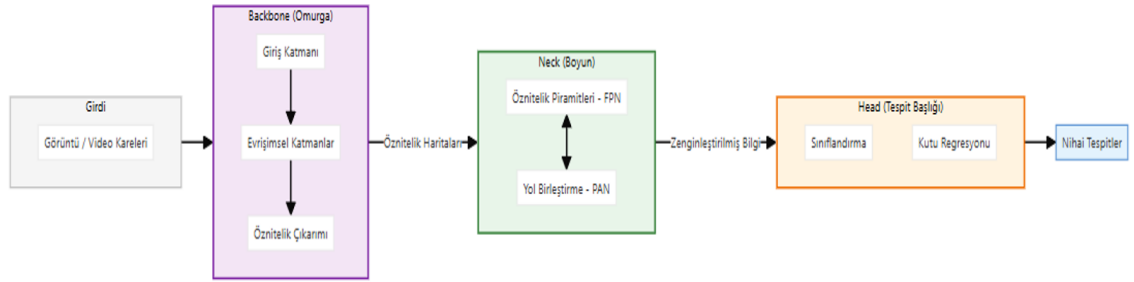


Şekil 2.4: Genel YOLO Mimarisi

Backbone (Omurga): Sistemin temel öznetelik çıkarım katmanıdır [1]. Girdi olarak alınan ham görüntülerden (RGB piksel verisi), evrimsel katmanlar aracılığıyla kenar, köşe, doku ve daha karmaşık yapısal özellikler çıkarılır. Projemizde kullanılan drone ve kuş nesnelerinin ayırt edici yapısal özellikleri (örneğin kanat formu ve gövde yapısı) bu aşamada çok ölçekli öznetelik haritalarına (feature maps) dönüştürülmektedir.

Neck (Boyun): Farklı katmanlardan elde edilen özneteliklerin birleştirildiği ve zenginleştirildiği ara katmandır. Bu aşamada kullanılan Feature Pyramid Network

(FPN) ve Path Aggregation Network (PAN) yapıları sayesinde farklı çözünürlük seviyelerindeki öznitelikler bir araya getirilir. Böylece hem uzak ve küçük boyutlu hedefler hem de yakın ve büyük nesneler aynı anda etkin şekilde tespit edilebilir. Bu durum, özellikle gökyüzünde küçük bir nokta olarak görülen uzak bir İHA'nın algılanması açısından kritik öneme sahiptir [2]. YOLO mimarisini oluşturan temel bileşenlerin operasyonel görev dağılımı ve veri akışı Şekil 2.5'de gösterilmektedir.



Şekil 2.5: Backbone, Neck ve Head Bileşenlerinin Operasyonel Yapısı

Head(Tespit Başlığı): Modelin karar verme katmanıdır. Neck'ten gelen zenginleştirilmiş öznitelikleri kullanarak nesnenin var olma olasılığını (objectness), sınıfını (örneğin kuş veya drone) ve konumunu (bounding box koordinatları) tahmin eder. Bu aşamada üretilen güven skorları doğrultusunda, çakışan tahminlerin elenmesi için Non-Maximum Suppression (NMS) uygulanır ve nihai tespit sonuçları elde edilir.

Bu üçlü yapının senkronize çalışması, modelin hesaplama maliyetini (computational cost) ve gecikmesini (latency) optimize ederek Raspberry Pi 5 gibi gömülü sistemler üzerinde gerçek zamanlı çıkarım yapılabilmesini mümkün kılmaktadır.

2.3.2 YOLOv8

YOLOv8, Ultralytics tarafından geliştirilen ve nesne tespiti alanında yaygın olarak kullanılan modern bir derin öğrenme modelidir. Önceki YOLO sürümlerine kıyasla daha esnek ve modüler bir mimari sunan YOLOv8; nesne tespiti (object detection), görüntü sınıflandırma (classification) ve görüntü bölütleme (segmentation) gibi farklı görevleri aynı altyapı üzerinden gerçekleştirebilmektedir. Bu çok yönlü yapı, modelin farklı uygulama senaryolarına kolaylıkla uyarlanabilmesini sağlamaktadır.

YOLOv8'in öne çıkan yapısal özelliklerinden biri, anchor-free (çapa bağımsız) tespit yaklaşımını benimsemesidir [1]. Geleneksel anchor-based yöntemlerde kullanılan önceden tanımlanmış referans kutular yerine, YOLOv8 doğrudan nesne konumlarını ve boyutlarını tahmin etmektedir. Bu yaklaşım, modelin farklı ölçeklerdeki nesneleri daha esnek bir şekilde öğrenmesine ve daha yüksek doğruluk oranlarına ulaşmasına olanak tanımaktadır. Ayrıca, daha sade bir yapı sunarak hesaplama karmaşıklığını da azaltmaktadır.

Bu çalışmada, hedef donanım olarak kullanılan Raspberry Pi 5'in sınırlı işlem gücü ve bellek kapasitesi dikkate alınarak, YOLOv8 model ailesinin en hafif ve hızlı versiyonu olan YOLOv8n (nano) tercih edilmiştir. YOLOv8n modeli, düşük hesaplama maliyetine rağmen yeterli doğruluk seviyesini koruyabilmesi sayesinde gömülü sistemler üzerinde gerçek zamanlı çıkarım (real-time inference) yapılabilmesi için uygun bir çözüm sunmaktadır.

2.3.3 YOLO26

Bu çalışmada kullanılan YOLO26 modeli, YOLO mimarisinin güncel yaklaşımlarını temel alan ve performans-verimlilik dengesi gözetilerek optimize edilmiş bir nesne tespit modelidir. Modelin geliştirilmesindeki temel amaç; mimari karmaşıklığı azaltarak hesaplama maliyetini düşürmek ve aynı zamanda tespit doğruluğunu artırmaktır.

YOLO26, önceki YOLO tabanlı modellere kıyasla daha sadeleştirilmiş bir ağ yapısına sahiptir. Model içerisinde gerçekleştirilen yapısal optimizasyonlar sayesinde, derin öğrenme mimarisindeki gereksiz veya tekrarlı işlem katmanları azaltılmıştır. Bu yaklaşım, modelin daha düşük parametre sayısı ile çalışmasını sağlayarak hem bellek kullanımını hem de işlem yükünü önemli ölçüde azaltmaktadır. Böylece model, özellikle sınırlı donanım kaynaklarına sahip sistemlerde daha verimli bir şekilde çalışabilmektedir.

Bu hafif ve optimize edilmiş mimari, GPU desteği bulunmayan veya sınırlı işlem gücüne sahip gömülü sistemlerde önemli bir avantaj sunmaktadır. Bu kapsamda, çalışmada otonom hedef tespiti ve takip performansını değerlendirmek amacıyla YOLO26 model ailesinin en küçük varyantı olan YOLO26n (nano) tercih edilmiştir [1]. Bu model, düşük hesaplama maliyetine rağmen yeterli doğruluk seviyesini koruyarak Raspberry Pi 5 üzerinde gerçek zamanlı çıkarım yapılabilmesine olanak tanımaktadır.

2.3.4 YOLOv8n ve YOLO26n Karşılaştırması

YOLOv8n ve YOLO26n modellerinin doğruluk, model boyutu ve hesaplama maliyeti gibi temel performans kriterleri açısından karşılaştırılması Tablo 2.3’de sunulmuştur.

Tablo 2.3: YOLOv8n ve YOLO26n Modellerinin Karşılaştırılması

Özellik	YOLOv8n	YOLO26n
Yayın Yılı	2023	2026
mAP50-95 (COCO val)	%37.3	%39.6
Parametre Sayısı	3.2M	2.4M
Model Dosya Boyutu	~6.2 MB	~5.0 MB
Mimari Yapı	Anchor-free	Anchor-free
Optimizasyon Odağı	Genel Performans	Hafifletilmiş / CPU Verimliliği

Gömülü sistemler ve mobil cihazlar üzerinde çalışan yapay zeka uygulamalarında, parametre sayısı ve model dosya boyutu sistem performansını doğrudan etkileyen temel kriterler arasında yer almaktadır. Parametre sayısı, modelin öğrenme kapasitesini temsil etmekle birlikte aynı zamanda işlemci (CPU) üzerinde gerçekleştirilecek hesaplama yükünü de belirlemektedir. Bu bağlamda, parametre sayısının 3.2 M’den 2.4 M seviyesine düşmesi, modelin daha düşük hesaplama maliyeti ile çalışabileceğini göstermektedir.

Benzer şekilde, model dosya boyutunun küçülmesi (6.2 MB yerine 5.0 MB), özellikle sınırlı bellek kapasitesine sahip sistemlerde önemli bir avantaj sağlamaktadır. Daha küçük model boyutu, hem bellek kullanımını azaltmakta hem de modelin yüklenme süresini iyileştirerek gerçek zamanlı uygulamalarda performansa katkı sunmaktadır.

Tablo 2.3 incelendiğinde, YOLO26n modelinin daha düşük parametre sayısı ve model boyutuna sahip olmasına rağmen, YOLOv8n modeline kıyasla daha yüksek bir doğruluk değeri (%39.6 mAP50-95) sunduğu görülmektedir. Bu durum, model mimarisinde gerçekleştirilen optimizasyonların hem verimlilik hem de doğruluk açısından olumlu sonuçlar doğurabileceğini göstermektedir. Elde edilen bu bulgular, gömülü sistemler üzerinde gerçekleştirilecek nesne tespiti uygulamaları için farklı model seçeneklerinin performans-verimlilik dengesi açısından değerlendirilmesinin önemini ortaya koymaktadır.

2.4 Neural Network Computing Framework (NCNN)

NCNN, Tencent tarafından geliştirilen ve özellikle mobil ve gömülü platformlar için optimize edilmiş yüksek performanslı bir sinir ağı çıkarım (inference) çerçevesidir. Herhangi bir üçüncü taraf kütüphaneye bağımlılığı bulunmayan NCNN, doğrudan işlemci mimarisine (Arm Neon, x86 AVX vb.) yönelik düşük seviyeli optimizasyonlar içermesi sayesinde kısıtlı donanıma sahip cihazlarda dahi oldukça yüksek hızlarda çalışabilmektedir [5].

Bu çalışma kapsamında eğitilen YOLO modellerinin Raspberry Pi 5 üzerinde gerçek zamanlı olarak çalıştırılabilmesi amacıyla NCNN formatına dönüştürülmesi tercih edilmiştir. NCNN'in tercih edilme nedenleri; düşük bellek kullanımı, Arm mimarisindeki işlemciler için sunduğu özel montaj (assembly) optimizasyonları ve çok çekirdekli paralel işleme kapasitesidir. Normal şartlarda yüksek işlem gücü gerektiren derin öğrenme modelleri, NCNN sayesinde Raspberry Pi 5 üzerinde yüksek kare hızı (FPS) değerlerinde sonuç üretebilmektedir. Derin öğrenme modelinin eğitimden gömülü sistem üzerinde çalıştırılabilir hale getirilmesine kadar geçen dönüşüm süreci Şekil 2.6'da gösterilmektedir.



Şekil 2.6:Derin Öğrenme Modeli Dönüştürme Süreci

2.5 Açık Kaynak Bilgisayarlı Görü Kütüphanesi (OpenCV)

OpenCV (Open Source Computer Vision Library), bilgisayarla görü ve görüntü işleme alanında kullanılan en yaygın açık kaynaklı kütüphanedir [6]. Proje kapsamında OpenCV; kamera görüntülerinin anlık olarak alınması, görüntülerin model için uygun çözünürlüğe ölçeklendirilmesi ve işlenen sonuçların (tespit kutuları, HUD arayüzü vb.) ekran üzerinde görselleştirilmesi aşamalarında kritik bir rol oynamaktadır.

Raspberry Pi 5 üzerinde C++ ve Python dilleriyle yüksek uyumlu çalışan OpenCV, görüntü akışını verimli şekilde işleyerek yapay zeka modeline girdi sağlar. Ayrıca, hedefin ekran üzerindeki koordinatlarının hesaplanması ve HUD üzerindeki nişangahın (crosshair) dinamik olarak çizilmesi gibi grafiksel işlemler de OpenCV fonksiyonları aracılığıyla gerçekleştirilmiştir. OpenCV'nin görüntü işleme uygulamalarındaki kullanımına dair örnek bir gösterim Şekil 2.7'de sunulmaktadır.



Şekil 2.7: OpenCV ve Python kullanımı

2.6 Python Programlama Dili

Python, yapay zeka ve veri bilimi alanında geniş kütüphane desteği ve okunabilir sözdizimi nedeniyle tercih edilmiştir. Bu projede, görüntü işleme hattının (pipeline) kontrolü, YOLO modellerinin yönetimi ve HUD arayüzünün mantıksal katmanları Python dili ile kodlanmıştır. Python'un sunmuş olduğu zengin ekosistem, farklı donanım ve yazılım bileşenlerinin entegrasyon sürecini hızlandırmış ve sistemin otonom karar verme yeteneklerinin geliştirilmesini kolaylaştırmıştır.

2.7 Roboflow

Roboflow, bilgisayarla görme (computer vision) uygulamaları için veri seti yönetimi, görüntü etiketleme ve veri artırma (augmentation) süreçlerini kolaylaştıran bulut tabanlı bir platformdur [7]. Platform, farklı kaynaklardan elde edilen görüntülerin düzenlenmesi, etiketlenmesi ve makine öğrenmesi modellerine uygun formatlara dönüştürülmesini sağlayarak veri hazırlama sürecini önemli ölçüde hızlandırmaktadır.

Bu çalışma kapsamında, drone ve kuş sınıflarına ait görüntülerin etiketlenmesi Roboflow platformu üzerinden gerçekleştirilmiştir. Etiketleme sürecinin ardından veri seti, model eğitimi için uygun olacak şekilde eğitim, doğrulama ve test (train–validation–test) alt kümelerine ayrılmıştır. Ayrıca, veri setinin genelleme yeteneğini artırmak amacıyla döndürme, ölçekleme, parlaklık değişimi ve gürültü ekleme gibi veri artırma teknikleri yine bu platform üzerinden uygulanmıştır.

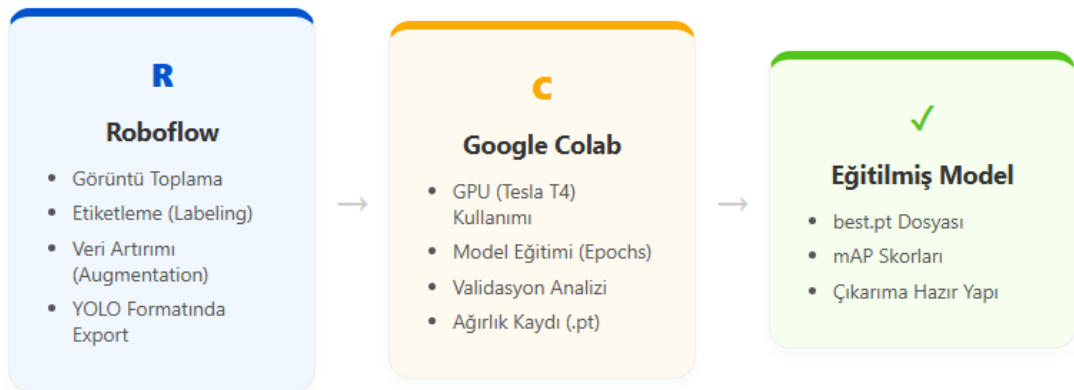
Elde edilen veri seti, YOLO mimarisine uygun formatta dışa aktarılmış ve model eğitimi sürecinde kullanılmıştır. Bu sayede, veri hazırlama süreci standartlaştırılmış ve model performansını artırmaya yönelik daha dengeli bir veri dağılımı elde edilmiştir.

2.8 Google Colab

Google Colab (Colaboratory), Google tarafından sunulan ve tarayıcı üzerinden Python tabanlı kodların yazılmasına ve çalıştırılmasına olanak sağlayan bulut tabanlı bir geliştirme ortamıdır [2]. Jupyter Notebook altyapısını temel alan bu platform, kullanıcıların herhangi bir yerel kurulum gerektirmeden yüksek hesaplama gücüne erişmesini mümkün kılmaktadır.

Bu çalışma kapsamında, derin öğrenme modellerinin eğitimi Google Colab ortamında gerçekleştirilmiştir. Raspberry Pi 5'in sınırlı işlem gücü ve donanım kaynakları göz önünde bulundurularak, model eğitimi için Colab platformunun sunduğu Grafik İşlem Birimi (GPU) desteğinden faydalanılmıştır. Bu sayede, eğitim süreleri önemli ölçüde azaltılmış ve daha karmaşık model yapılarının etkin bir şekilde eğitilmesi sağlanmıştır.

Ayrıca, Colab ortamı üzerinden veri setlerinin yönetimi, model eğitimi, performans değerlendirmesi ve sonuçların görselleştirilmesi süreçleri bütünsel bir şekilde yürütülmüştür. Bulut tabanlı yapısı sayesinde farklı cihazlardan erişim imkanı sunması, geliştirme sürecinin esnek ve verimli bir şekilde ilerlemesine katkı sağlamıştır. Veri seti oluşturma, model eğitimi ve çıkarım aşamalarını kapsayan uçtan uca iş akışı Şekil 2.8'de görsel olarak sunulmaktadır.



Şekil 2.8: Veri hazırlama, model eğitimi ve model çıktısı elde etme süreci

2.9 PCA9685 Servo Sürücü Kartı

PCA9685, I2C haberleşme protokolü üzerinden çalışan, 16 kanal çıkışa ve 12-bit çözünürlüğe sahip bir PWM (Pulse Width Modulation) sürücü entegresidir [10]. Bu kart, özellikle çok sayıda servo motorun hassas ve senkronize bir şekilde kontrol edilmesi gereken uygulamalarda yaygın olarak kullanılmaktadır.

Bu çalışma kapsamında PCA9685 kartı, otonom hedef takip sisteminde kullanılan servo motorların kontrolü amacıyla tercih edilmiştir. Servo motorlar, tespit edilen hedefin konumuna göre yönlendirilerek sistemin takip mekanizmasını gerçekleştirmektedir. PCA9685 kartı sayesinde bu motorların hassas açı kontrolü sağlanmış ve stabil bir hareket kabiliyeti elde edilmiştir.

Raspberry Pi 5 gibi tek kartlı bilgisayarlarda PWM sinyalinin doğrudan GPIO pinleri üzerinden üretilmesi, işlemci yüküne bağlı olarak sinyal kararsızlıklarına yol açabilmektedir. Bu durum, servo motorların titreşimli veya düzensiz hareket etmesine neden olabilir. PCA9685 kullanımı ile PWM sinyal üretimi harici bir donanıma devredilerek Raspberry Pi üzerindeki işlem yükü azaltılmış ve daha kararlı bir kontrol mekanizması sağlanmıştır.

Ayrıca, PCA9685'in sunduğu 12-bit çözünürlük, servo motorların çok küçük açı değişimlerine dahi hassas tepki verebilmesini mümkün kılmaktadır. Bu özellik, özellikle hedefin hassas şekilde merkezlenmesi gereken otonom takip sistemlerinde önemli bir avantaj sağlamaktadır. Bu çalışmada kullanılan PCA9685 servo sürücü kartının fiziksel yapısı Şekil 2.9'da sunulmaktadır.



Şekil 2.9: PCA9685 Servo Sürücü Kartı

2.10 Pan-Tilt Mekanizması

Pan-Tilt mekanizması, kamera modülüne yatay (pan) ve dikey (tilt) eksenlerde hareket kabiliyeti kazandıran, iki serbestlik dereceli (2 DOF) bir mekanik sistemdir. Bu yapı, otonom hedef takip aşamasında görüntü işleme algoritmalarından elde edilen koordinat bilgilerine bağlı olarak kameranın fiziksel olarak hedefe yönlendirilmesini sağlamaktadır. Pan-Tilt mekanizmasına ait fiziksel görünüm Şekil 2.10'da gösterilmektedir.



Şekil 2.10:Pan-Tilt Mekanizması

Mekanizma üzerinde bulunan iki adet servo motor, PCA9685 sürücü kartı aracılığıyla kontrol edilmektedir [10]. Bu sayede, servo motorların hassas ve kararlı bir şekilde konumlandırılması mümkün hale gelmiştir. Sistem, tespit edilen hedefin görüntü merkezine göre konumunu (ofset değerini) analiz ederek, bu sapmayı minimize edecek şekilde kamerayı ilgili eksenlerde hareket ettirmektedir.

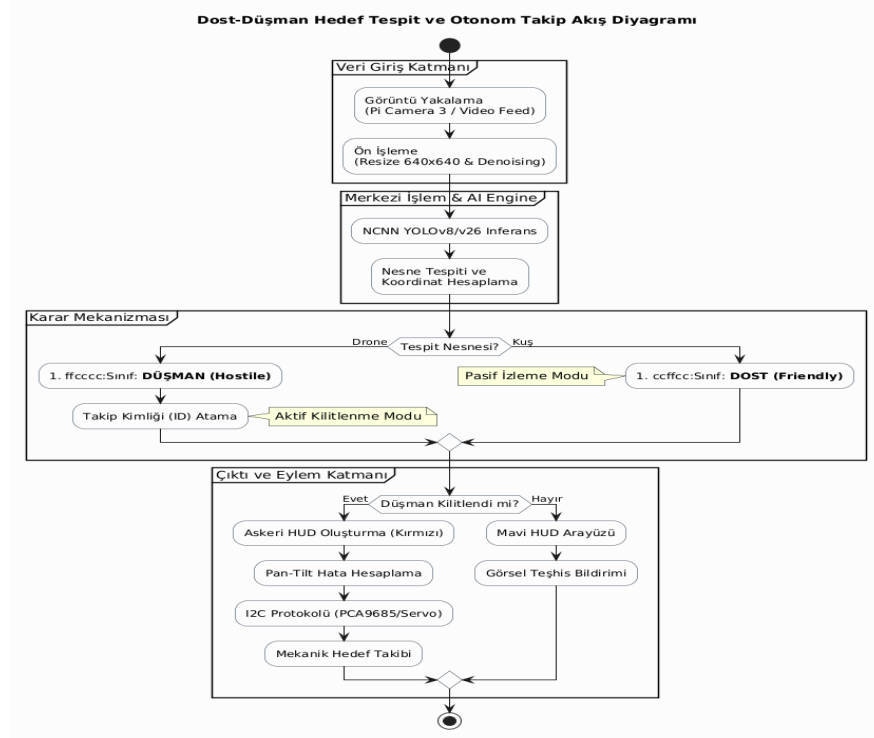
Bu yaklaşım, hedef nesnenin (drone veya kuş) kamera görüş alanı dışına çıkma olasılığını azaltarak, takibin yalnızca yazılımsal değil aynı zamanda fiziksel düzeyde de sürekliliğini sağlamaktadır.

3. SİSTEM TASARIMI VE OLUŞTURULMASI

Çalışmanın bu kısmında, yapay zeka destekli dost-düşman hedef tespit sisteminin gömülü sistem entegrasyonu, tasarım metodolojisi ve karşılaşılan teknik darboğazlara sunulan özgün çözüm yöntemleri incelenmektedir.

3.1 Sistemin Genel Çalışma Mantığı ve Akış Diyagramı

Sistemin bütünleşik mimarisi ve veri akış hiyerarşisi Şekil 3.1’de sunulmuştur. Şekil 3.1’de görüldüğü üzere sistem, dört ana fonksiyonel katmandan meydana gelmektedir. Akışın başlangıcını oluşturan Veri Giriş Katmanı, dış dünyadan ham verinin toplanmasından sorumludur. Toplanan bu veri, sistemin kalbi niteliğindeki Merkezi İşlem Birimi’nde ön işleme tabi tutularak analiz edilebilir formata dönüştürülür. Üçüncü aşamada yer alan Karar Mekanizması, elde edilen öznitelikleri sınıflandırarak dost ve düşman unsurları birbirinden ayırır. Akışın son halkası olan Çıktı ve Eylem Katmanı ise, bir önceki aşamadan gelen karara bağlı olarak sistemin fiziksel veya görsel tepki vermesini sağlar. Her bir katman kendi içerisinde özgün bir alt hiyerarşiye sahip olup, birbirleriyle tam senkronize biçimde çalışmaktadır



Şekil 3.1: Dost-Düşman Hedef Tespit ve Otonom Takip Diyagramı

3.1.1 Görüntü Giriş Katmanı ve Ön İşleme

Sistem, girdi olarak hem önceden kaydedilmiş video dosyalarını (.mp4) hem de Raspberry Pi Camera 3 Wide üzerinden gelen canlı görüntü akışını kabul edebilecek esnek bir girdi katmanına sahiptir. Gelen ham görüntü, Python tabanlı OpenCV ve 'libcamera' kütüphaneleri aracılığıyla yakalanır. Yakalama aşamasında, görüntüdeki yüksek frekanslı gürültülerin azaltılması ve yapay zeka modelinin ihtiyaç duyduğu standart çözünürlük değerlerine (örneğin 640x640 piksel) ölçeklendirilmesi (resizing) sağlanır. Bu ön işleme adımları, merkezi işlem birimi (CPU) üzerindeki gereksiz yükü minimize ederek, model çıkarım süresini kısaltmaktadır.

3.1.2 Yapay Zeka ve Karar Motoru (Merkezi İşlem)

Sistemin karar merkezi olarak işlev gören Raspberry Pi 5, NCNN çıkarım motorunu kullanarak veri setimizle eğitilmiş derin öğrenme modellerini çalıştırır. Süreç iki ana aşamadan oluşmaktadır:

- **Sınıflandırma Mantığı:** Model tarafından 'inference' anında tespit edilen her nesne, sistemin 'Dost-Düşman Hiyerarşisi' süzgecinden geçirilir. Bu kapsamda İHA (Drone) etiketli nesneler 'Düşman' (Hostile), Kuş (Bird) etiketli nesneler ise 'Dost' (Friendly) olarak tanımlanır.
- **Akıllı Takip ve Dinamik Kilitleme:** Sınıflandırma sonrası sistem bir eylem kararı üretir. Bir nesne 'Dost' olarak tanımlandığında sistem pasif izleme modunda (sadece görsel teşhis) kalırken; 'Düşman' tehdidi algılandığında sistem **Aktif Takip Modu**'na geçiş yapar. Bu modda hedefe bir takip kimliği (tracking ID) atanır ve nesnenin görüntü düzlemindeki merkez koordinat verileri hesaplanarak kontrol birimlerine aktarılır.

3.1.3 Sistem Çıktıları ve Mekanik Kontrol

İşleme ve karar aşamalarından elde edilen veriler, sistem tarafından biri görsel diğeri mekanik olmak üzere iki farklı kanaldan dış dünyaya aktarılır:

1. **Yazılımsal Çıktı** : İşlenmiş video akışı üzerine, OpenCV fonksiyonları kullanılarak askeri standartlarda sembolojiler (hedef çerçeveleri, dinamik vizörler, dost/düşman etiketleri) ve sistemin çalışma metrikleri (anlık FPS, hedef sayısı) bindirilir. Bu Heads-Up Display (HUD) arayüzü, operatöre anlık durumsal farkındalık sağlar.
2. **Donanımsal Çıktı** : Sadece 'Düşman' olarak işaretlenen ve takibe alınan hedeflerin merkez koordinatları (X, Y sapma değerleri), I2C haberleşme protokolü üzerinden PCA9685 servo sürücüsüne iletilir. Sürücü, gelen dijital veriyi analog PWM sinyallerine dönüştürerek Pan-Tilt mekanizmasını harekete geçirir. Bu fiziksel eylem sonucunda, hareketli hedef kameranın görüş alanında sürekli olarak merkezlenmiş (locked-on) olur.

3.2 Veri Seti Hazırlama ve Etiketleme Süreci

Derin öğrenme tabanlı hedef tespit modellerinin başarısı, doğrudan eğitildikleri veri setinin kalitesine, çeşitliliğine ve probleme uygunluğuna bağlıdır. Bu çalışmada, modelin hazır bir veri setiyle eğitilmesi yerine, projenin özgün gereksinimleri (dost-düşman ayrımı ve gökyüzü koşulları) doğrultusunda özel bir veri seti oluşturulmuştur.

3.2.1 Veri Kaynakları ve Ham Veri Toplama

Projede kullanılacak veri setinin temelini oluşturmak amacıyla, telifsiz (royalty-free) medya platformları olan iStock, Shutterstock ve Pixabay üzerinden yüksek çözünürlüklü uçsuz bucaksız gökyüzü, ormanlık alan ve bina arka planlarına sahip çeşitli drone ve kuş videoları indirilmiştir. Modelin farklı drone tiplerini ve sürü konseptlerini de tanıyabilmesi için Kaggle ve GitHub üzerinde paylaşılan açık erişimli drone takip veri setlerinden (örneğin; "drone-tracking-datasets") faydalanılmıştır. Bu aşamada temel amaç, modelin ezberlemesini (overfitting)

önlemek için veri havuzunu farklı aydınlatma, hava durumları ve kamera açılarından oluşan görüntülerle olabildiğince çeşitlendirmektir.

Toplanan videolar; farklı uçuş açıları, mesafeler, arka planlar ve ışık koşullarını içerecek şekilde seçilmiştir. Özellikle şehir içi, doğal ortamlar ve farklı hava koşullarında çekilmiş görüntüler tercih edilerek modelin genelleme yeteneğinin artırılması hedeflenmiştir. Bu yaklaşım, gerçek dünya senaryolarını daha iyi temsil eden bir veri dağılımı elde edilmesini sağlamıştır.

Ham veri olarak elde edilen videolar, doğrudan kullanılmak yerine belirli bir kare hızında (frames per second – FPS) parçalanarak görüntü formatına dönüştürülmüştür. Bu işlem sayesinde, tek bir videodan farklı zaman anlarına ait çok sayıda örnek elde edilmiştir. Ayrıca, ardışık kareler arasındaki benzerliği azaltmak amacıyla belirli aralıklarla örnekleme yapılmış ve veri setindeki tekrar oranı düşürülmüştür. Veri setinin çeşitliliğini ve farklı çevresel koşulları yansıtan örnek görüntüler Şekil 3.2’de sunulmaktadır.



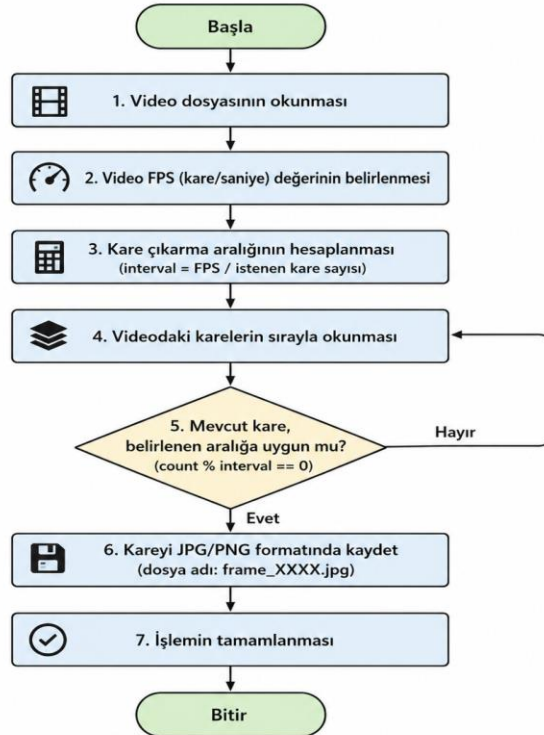
Şekil 3.2: Örnek Veri Görüntüleri

3.2.2 Video Kayıtlarından Manuel Kare Çıkarımı

Ham veri olarak elde edilen video kayıtları, model eğitimi için doğrudan kullanılmak yerine kare bazlı görüntülere dönüştürülmüştür. Bu işlem, video içerisindeki zamansal bilgiyi ayrıştırarak her bir anı bağımsız bir eğitim örneği haline getirmeyi amaçlamaktadır. Bu kapsamda, videolar belirli bir kare hızında (frames per second – FPS) örneklenerek görüntü setine dönüştürülmüştür.

Kare ayıklama işlemi sırasında, veri setinde aşırı benzer görüntülerin oluşmasını engellemek amacıyla düşük FPS değerleri tercih edilmiştir. Bu çalışmada, videolar yaklaşık olarak 2–5 FPS aralığında örneklenmiş olup, bu sayede ardışık kareler arasındaki benzerlik azaltılarak veri setinin çeşitliliği artırılmıştır. Bu yaklaşım, modelin aşırı öğrenme (overfitting) eğilimini azaltmakta ve genelleme performansını olumlu yönde etkilemektedir.

Kare ayıklama işlemi, Python programlama dili kullanılarak geliştirilmiş bir betik (script) aracılığıyla gerçekleştirilmiştir. Bu betikte, video dosyaları okunarak belirlenen aralıklarda kareler alınmış ve standart görüntü formatında (örneğin .jpg veya .png) diske kaydedilmiştir. İşlem sırasında, tüm görüntüler belirli bir isimlendirme düzenine göre saklanarak veri setinin düzenli ve izlenebilir olması sağlanmıştır. Video kayıtlarından kare çıkarma sürecinin işlem adımları Şekil 3.3'te gösterilmiştir.



Şekil 3.3: Video kayıtlarından kare çıkarma sürecinin akış şeması

Geliştirilen betik, her video için toplam kare sayısını analiz ederek, seçilen FPS değerine uygun şekilde örnekleme yapmakta ve gereksiz tekrarların önüne

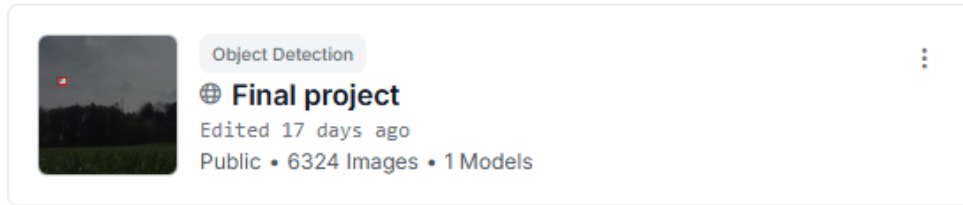
geçmektedir. Ayrıca, veri seti oluşturma sürecinde tutarlılığı sağlamak amacıyla tüm videolar aynı parametreler kullanılarak işlenmiştir. Bu sayede, eğitim veri seti homojen bir yapıya kavuşturulmuş ve model performansını olumsuz etkileyebilecek veri dengesizlikleri minimize edilmiştir.

Bu süreç sonucunda elde edilen kareler, bir sonraki aşamada etiketleme ve veri artırma işlemlerine tabi tutulmak üzere hazır hale getirilmiştir.

3.2.3 Veri Seti Genel Özellikleri

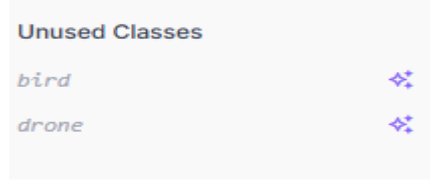
Bu çalışma kapsamında oluşturulan veri seti, video verilerinden elde edilen karelerin işlenmesi ve etiketlenmesi sonucunda oluşturulmuştur. Veri seti, Roboflow platformu kullanılarak düzenlenmiş ve yönetilmiştir.

Toplamda yaklaşık 6324 adet görüntü, veri setinin ham halini oluşturmakta olup, veri artırma (augmentation) işlemleri sonrasında bu sayı 10752 görüntüye çıkarılmıştır. Bu durum, modelin farklı senaryolara karşı daha dayanıklı hale getirilmesi ve genelleme yeteneğinin artırılması açısından önemli bir katkı sağlamaktadır. Oluşturulan veri setinin Roboflow platformu üzerindeki genel görünümü Şekil 3.4'te sunulmaktadır.



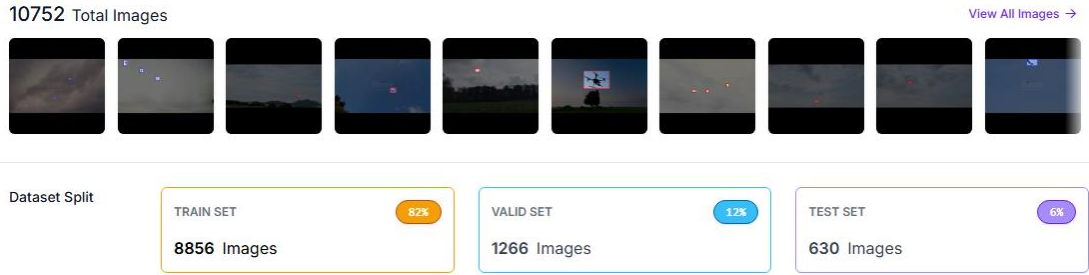
Şekil 3.4: Veri setinin Roboflow platformu üzerindeki görünümü

Veri seti, iki temel sınıf içerecek şekilde hazırlanmıştır: drone ve kuş. Bu sınıflar, çalışmanın hedefi doğrultusunda dost ve düşman hedeflerin ayırt edilmesini sağlayacak şekilde belirlenmiştir. Veri setinde tanımlanan sınıf yapısı Şekil 3.5'te gösterilmektedir.



Şekil 3.5: Veri setinde kullanılan sınıflar (drone ve kuş)

Veri seti, model eğitimi sürecinde kullanılmak üzere eğitim (train), doğrulama (validation) ve test (test) olmak üzere üç alt kümeye ayrılmıştır. Bu kapsamda veri dağılımı yaklaşık olarak %82 eğitim (8856 görüntü), %12 doğrulama (1266 görüntü) ve %6 test (630 görüntü) olacak şekilde gerçekleştirilmiştir. Model eğitimi sürecinde kullanılan veri setinin eğitim, doğrulama ve test alt kümelerine ayrılma oranları Şekil 3.6’da görsel olarak sunulmaktadır.



Şekil 3.6: Veri setinin eğitim, doğrulama ve test kümelerine dağılımı

Görüntüler, modelin giriş boyutlarına uyum sağlayacak şekilde farklı çözünürlüklerde yeniden boyutlandırılmıştır. Bu çalışmada, özellikle 640×640 çözünürlük yaygın olarak kullanılmış, ancak bazı veri seti versiyonlarında farklı boyutlandırma yaklaşımları da değerlendirilmiştir. Bu esnek yapı, model performansının farklı giriş boyutlarına göre analiz edilmesine olanak tanımaktadır.

Veri setinin çeşitliliğini artırmak amacıyla yatay çevirme (horizontal flip), parlaklık (brightness) ve renk tonu (hue) değişimleri gibi veri artırma teknikleri uygulanmıştır. Bu sayede modelin farklı ışık koşulları ve çevresel değişimlere karşı daha dayanıklı hale gelmesi hedeflenmiştir. Veri setinin ön işleme ve artırma süreçlerine ait parametreler Şekil 3.7’de sunulmaktadır.

Preprocessing	Auto-Orient: Applied Resize: Fit (black edges) in 960x960
Augmentations	Outputs per training example: 2 Flip: Horizontal Hue: Between -5° and +5° Brightness: Between -15% and +15% Exposure: Between -10% and +10% Blur: Up to 0.5px

Şekil 3.7: Veri setine uygulanan ön işleme ve veri artırma ayarları

3.2.4 Veri Etiketleme Süreci (Roboflow)

Yapay sinir ağlarının başarısı, büyük ölçüde gözetimli öğrenme (supervised learning) sürecinde kullanılan verilerin doğruluğuna bağlıdır. Bu doğrultuda, karmaşık gökyüzü koşullarında (farklı bulut yoğunlukları, ışık değişimleri ve uzak mesafe görüntüleri) kuş ve drone ayırımını gerçekleştirebilmek amacıyla titiz bir manuel etiketleme süreci yürütülmüştür.

Etiketleme platformu olarak bulut tabanlı Roboflow ekosistemi tercih edilmiştir [7]. Roboflow’un tercih edilmesinin temel nedenleri; çok kullanıcıli etiketleme desteği, YOLO formatında doğrudan dışa aktarım imkânı ve veri seti üzerindeki sınıf dağılımını (class balance) analiz edebilme yeteneğidir.

Etiketleme sürecinde her bir nesne, “Bounding Box” (sınırlayıcı kutu) yöntemi kullanılarak işaretlenmiştir. Bu yöntemde, nesneler mümkün olan en dar sınırlar içerisinde çevrelenerek modelin nesne sınırlarını arka plan gürültüsünden ayırt edebilmesi hedeflenmiştir. Veri setinde gerçekleştirilen etiketleme işlemlerine ait örnek görüntüler Şekil 3.8’de sunulmaktadır.



Şekil 3.8: Etiketlenmiş Drone ve Kuş Görüntüleri

3.2.5 Zorlayıcı arka planlar ve etiketsiz (negatif) örnekler

Gerçek ortamda gökyüzü ve açık alan görüntülerinde ağaç dalları, ince yapılar ve yüksek kontrastlı dokular gibi unsurlar, küçük hedeflerle görsel olarak benzer çizgiler oluşturabildiğinden model tarafından yanlışlıkla hedef olarak algılanma riski taşır.

Bu nedenle veri setine, hedef içermeyen ancak benzer görsel karmaşıklığa sahip sahneler de dahil edilmiştir. Bu tür görüntülerde bilinçli olarak etiket (bounding box) bırakılmamış; böylece modelin arka plan dokularını “hedef” olarak öğrenmesinin önüne geçilmesi ve yanlış pozitif (false positive) oranının azaltılması amaçlanmıştır. Modelin yanlış pozitif üretimini azaltmak amacıyla veri setine eklenen negatif örneklere ait görüntüler Şekil 3.9’da görsel olarak sunulmaktadır.



Şekil 3.9: Etiketsiz Görüntüler

Bu yaklaşım, özellikle düşük çözünürlükte veya hedefin çok küçük görüldüğü karelerde sınırlayıcı unsurların (dallar, hatlar vb.) drone/kuş ile karıştırılmasını engellemeye yönelik bir negatif örnek (background / no-object) stratejisi olarak değerlendirilmektedir.

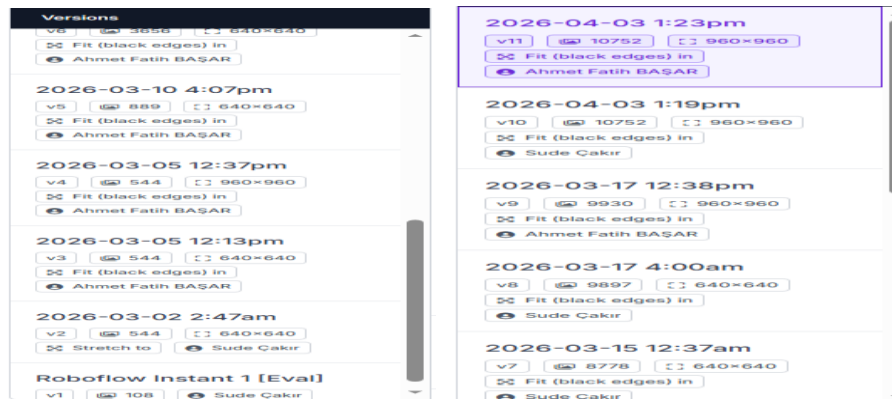
3.2.6 Veri setinin sürüm bazlı geliştirilmesi

Veri seti, başlangıçta sınırlı sayıda görüntüyle oluşturulmuş ve Roboflow üzerinde sürüm (version) mantığıyla zaman içinde genişletilmiştir. İlk aşamalarda küçük

ölçekli değerlendirme kümeleriyle başlanmış; ardından veri sayısı artırılarak farklı sahnelerin ve koşulların veri setine dahil edilmesi hedeflenmiştir.

Bu süreçte yalnızca görüntü adedinin artırılması değil, yeniden boyutlandırma stratejilerinin (ör. en-boy oranını koruyan “fit/black edges” yaklaşımı) ve giriş çözünürlüğünün (ör. 640×640 ve 960×960 gibi) denenmesi de gerçekleştirilmiştir. Özellikle erken sürümlerde kullanılan “stretch” benzeri yaklaşımların görüntüde bozulmalara yol açabileceği düşünülerek, sonraki sürümlerde oran koruyan ön işleme yöntemlerine geçilmiştir. Veri setinin genişletilmesi ve sürümler arası düzenlemeler; saha koşullarında çekilen yeni örneklerin eklenmesinin yanı sıra, eğitilen modellerin doğruluk–geri çağırma performansı, yanlış pozitif/yanlış negatif gözlemleri ve tipik hata türleri (ör. küçük hedefler, düşük kontrast, karmaşık arka planlar) doğrultusunda planlanmıştır.

Böylece veri seti, tek seferde sabitlenmiş bir koleksiyon olmak yerine, proje boyunca ölçülebilir geri bildirimlerle güncellenen iteratif bir yapı kazanmıştır. Son aşamalarda görüntü sayısının binlerce örneğe ulaştığı ve seçilen sürümde (ör. v11) eğitim–doğrulama–test bölünmesi ile birlikte ön işleme ve artırma adımlarının da tutarlı biçimde uygulandığı görülmektedir. Veri setinin versiyonlama süreci ve farklı sürümlere ait kayıtlar Şekil 3.10’da sunulmaktadır.



Şekil 3.10: Veri setinin versiyonlama ve güncelleme süreci

4. MODELLEME VE DENEYSEL ÇALIŞMALAR

4.1 Problem biçimlendirmesi

Bu çalışmada ele alınan problem, görüntü veya video kareleri üzerinde gökyüzü ve açık alan sahnelerinde bulunan hedef nesnelerin otomatik olarak tespit edilmesi ve sınıflandırılmasıdır. Problemin doğası gereği, klasik sınıflandırma (classification) yaklaşımından ziyade nesne tespiti (object detection) yöntemi tercih edilmiştir. Çünkü bu problemde yalnızca nesnenin hangi sınıfa ait olduğu değil, aynı zamanda görüntü içerisindeki konumunun da belirlenmesi gerekmektedir [4].

Bu kapsamda her bir hedef nesne, görüntü uzayında iki boyutlu koordinatlar ile tanımlanan bir sınırlayıcı kutu (bounding box) ile temsil edilmektedir. Model çıktısı, her bir nesne için sınıf etiketi (örneğin “kuş” veya “drone”) ve buna karşılık gelen güven skoru (confidence score) ile birlikte üretilmektedir. Güven skoru, modelin yaptığı tahminin doğruluğuna ilişkin olasılıksal bir ölçü sunmaktadır [2]. Eğitilen modelin drone ve kuş sınıflarına yönelik gerçekleştirdiği nesne tespitine ait örnek çıktı Şekil 4.1’de görsel olarak sunulmaktadır.



Şekil 4.1: İki sınıflı tespit örneği: kuş ve drone için sınır kutuları

Şekil 4.1’da örnek bir nesne tespiti çıktısı verilmiştir. Görüldüğü üzere model, giriş görüntüsü üzerinde birden fazla nesneyi eş zamanlı olarak tespit edebilmekte ve her bir nesneyi sınırlayıcı kutular ile işaretlemektedir. Bu yaklaşım, özellikle gerçek

zamanlı hedef takibi ve dost-düşman ayrımı gibi uygulamalarda kritik bir öneme sahiptir.

Bu çalışmada problem, iki sınıflı bir nesne tespiti problemi olarak ele alınmıştır: “kuş” (dost) ve “drone” (düşman). Bu sınıflandırma, sistemin gerçek dünya senaryolarında karar verme mekanizmasını basitleştirmek ve performansını artırmak amacıyla belirlenmiştir.

Tanımlanan bu problem çerçevesinde, yüksek doğruluk ve gerçek zamanlı performans sağlayabilecek derin öğrenme tabanlı nesne tespiti modelleri tercih edilmiştir. Bu kapsamda, çalışmada kullanılan YOLO tabanlı modellerin mimarisi ve eğitim süreci bir sonraki bölümde detaylı olarak ele alınmaktadır.

4.2 Yolo Modelleri ve Eğitim ve Süreci

Nesne tespiti alanında son yıllarda en yaygın kullanılan yaklaşımlardan biri olan YOLO (You Only Look Once) mimarisi, yüksek hız ve doğruluk dengesi sayesinde gerçek zamanlı uygulamalarda ön plana çıkmaktadır. YOLO modelleri, görüntü üzerinde tek geçişte (single-stage) hem nesne konumunu hem de sınıfını tahmin edebilen bir yapıya sahiptir.

Bu çalışmada, YOLOv8n modeli temel nesne tespiti algoritması olarak seçilmiş ve modelin eğitimi, optimizasyonu ve performans analizi detaylı olarak incelenmiştir.

4.2.1 Eğitim Ortamı ve Hiperparametreler

Derin öğrenme tabanlı modellerin eğitimi, yüksek hesaplama kapasitesi (FLOPs) gerektirdiğinden, bu süreç doğrudan bir uç cihaz olan Raspberry Pi 5 üzerinde gerçekleştirilmemiştir. Bunun yerine eğitim işlemleri, Google Colab bulut platformu üzerinde sağlanan NVIDIA L4 ve A100-SXM4 grafik işlem birimleri (GPU) kullanılarak yürütülmüştür.

Eğitim aşamasında, modelin görsel özellikleri (kenar, köşe ve şekil gibi temel yapısal bileşenler) daha hızlı öğrenbilmesi amacıyla sıfırdan eğitim yaklaşımı tercih

edilmemiştir. Bunun yerine, hesaplama maliyetini azaltan ve eğitim sürecini hızlandıran transfer öğrenimi (Transfer Learning) yöntemi uygulanmıştır [9]. Bu doğrultuda, COCO veri seti üzerinde önceden eğitilmiş YOLOv8n modeline ait ağırlıklar (yolov8n.pt) başlangıç parametreleri olarak kullanılmıştır.

Model performansının veri seti kalitesine olan etkisini analiz edebilmek amacıyla, farklı eğitim aşamalarında hiperparametreler sabit tutulmuştur. Özellikle uzak mesafede bulunan küçük ölçekli nesnelerin tespit edilebilirliğini artırmak amacıyla giriş görüntü boyutu **imgsz=960** olarak seçilmiştir [1]. Ayrıca eğitim sürecinde iterasyon sayısı **epochs=100** ve veri yığını boyutu **batch size=16** olarak belirlenmiştir.

4.2.2 Veri Seti ile Optimizasyon ve İteratif Başarım Artışı

Bölüm 3.2’de belirtildiği üzere, bu çalışmada kullanılan veri seti statik bir yapı olarak ele alınmamış, iteratif biçimde güncellenerek versiyonlanmıştır. Veri kalitesinin nesne tespit performansı üzerindeki etkisini analiz edebilmek amacıyla, YOLOv8n modeli hem ilk aşama (v5) hem de nihai aşama (v10) veri setleri üzerinde aynı hiperparametreler ile eğitilmiştir.

İlk veri seti versiyonu (v5) ile gerçekleştirilen deneylerde, modelin özellikle “kuş” (dost) ve “drone” (düşman) sınıfları arasında belirgin karışıklıklar yaşadığı ve genel doğruluk seviyesinin sınırlı kaldığı gözlemlenmiştir. Bu doğrultuda veri seti; hedef odaklı veri toplama, veri artırma ve etiketleme iyileştirme süreçleri ile yeniden düzenlenmiş ve nihai v10 versiyonu oluşturulmuştur. Veri seti kalitesinin model performansı üzerindeki etkisini ortaya koymak amacıyla elde edilen karşılaştırmalı başarım metrikleri Tablo 4.1’de sunulmaktadır.

Tablo 4.1: Modellerin Başarım Metrikleri

Performans Metriği	İlk Eğitim (v5 Veri Seti)	Final Eğitimi (v10 Veri Seti)	İyileşme / Değişim
Doğrulama (Validation) Verisi	177 Görüntü	1266 Görüntü	+1089 Görüntü
Genel mAP50	% 90.9	% 97.9	+ % 7.0
Genel mAP50-95 (Zorlu Metrik)	% 49.2	% 67.4	+ % 18.2
Kuş Sınıfı Kesinlik (Precision)	% 79.8	% 93.9	+ % 14.1
Kuş Sınıfı Duyarlılık (Recall)	% 83.8	% 94.1	+ % 10.3
Drone Sınıfı Kesinlik (Precision)	% 92.2	% 96.9	+ % 4.7
Drone Sınıfı Duyarlılık (Recall)	% 93.0	% 96.2	+ % 3.2

Her iki veri seti üzerinde elde edilen performans metrikleri Tablo 4.1’te karşılaştırmalı olarak sunulmaktadır. Sonuçlar incelendiğinde, model mimarisi ve hiperparametrelerde herhangi bir değişiklik yapılmaksızın yalnızca veri setinin iyileştirilmesi ile mAP50-95 metriğinde **%18.2** oranında önemli bir artış sağlandığı görülmektedir. Ayrıca, kuş sınıfına ait kesinlik (precision) değerinde elde edilen **%14.1’lik** iyileşme, sistemin dost-düşman ayrımındaki başarımını belirgin şekilde artırmıştır.

Elde edilen bulgular, veri odaklı yapay zeka (data-centric AI) yaklaşımının, özellikle sınırlı donanım kaynaklarına sahip sistemlerde model performansını artırmada kritik bir rol oynadığını göstermektedir.

4.2.3 YOLOv8n ve YOLO26n Sonuçları Üzerinden Mimari Karşılaştırma

Nihai model seçiminin yapılabilmesi amacıyla, YOLO serisinin yaygın olarak kullanılan YOLOv8n modeli ile daha güncel mimari özellikler sunan YOLO26n modeli arasında eşdeğer koşullarda bir karşılaştırma (benchmark) gerçekleştirilmiştir. Her iki model de aynı veri seti (1266 doğrulama görüntüsü) ve sabit hiperparametreler ile eğitilmiştir. Elde edilen doğruluk ve donanım maliyeti metrikleri Tablo 4.2’de özetlenmektedir.

Tablo 4.2: YOLOv8n ve YOLO26n Karşılaştırması

Performans Metriği	YOLOv8n	YOLO26n	Karşılaştırma / Fark
Doğrulama (Validation) Verisi	1266 Görüntü	1266 Görüntü	Eşdeğer Şartlar
Genel mAP50-95 (Zorlu Metrik)	% 67.4	% 67.4	Tamamen Eşit
Genel mAP50	% 97.9	% 97.7	Neredeyse Eşit
Parametre Sayısı	~ 3.01 Milyon	~ 2.37 Milyon	% 21 Düşüş (YOLO26n Lehine)
Model Dosya Boyutu (.pt)	~ 6.2 MB*	~ 4.8 MB*	Daha Hafif Model (YOLO26n Lehine)

Tablo 4.2 incelendiğinde, her iki modelin de en zorlu performans metriği olan mAP50-95 açısından **%67.4** seviyesinde eşdeğer bir doğruluk sağladığı görülmektedir. Benzer şekilde mAP50 değerlerinde de önemli bir fark bulunmamaktadır. Buna karşın, YOLO26n modelinin parametre sayısında yaklaşık **%21** oranında azalma sağladığı ve model dosya boyutunun daha düşük olduğu gözlemlenmiştir. Bu durum, özellikle gömülü sistemlerde bellek kullanımı açısından YOLO26n modeline teorik bir avantaj kazandırmaktadır.

Ancak saha uygulamaları ve sistem entegrasyonu açısından yapılan değerlendirmelerde, yalnızca model boyutu değil, aynı zamanda yazılım ekosistemi uyumluluğu ve kararlılığı da belirleyici faktörler olarak öne çıkmaktadır. Bu kapsamda, YOLOv8 mimarisinin literatürde yaygın olarak kabul görmesi, geniş açık kaynak desteğine sahip olması ve özellikle OpenCV ile NCNN gibi gömülü sistem araçlarıyla yüksek uyumluluk göstermesi önemli bir avantaj sağlamaktadır.

Buna karşılık, YOLO26n mimarisinin gömülü sistem ortamlarında henüz yeterli olgunluğa ulaşmamış olması, potansiyel entegrasyon ve kararlılık risklerini beraberinde getirmektedir. Bu nedenle, doğruluk açısından benzer performans sunmasına rağmen daha güvenilir ve kararlı bir altyapı sağlaması sebebiyle nihai model olarak YOLOv8n tercih edilmiştir.

4.3 Hedef Tespiti ve Kararlı Takip Yaklaşımı

Bu sistemin temel hedefi, drone sınıfına ait hava araçlarının (hedef/düşman) gerçek zamanlı olarak tespit edilmesi ve süreklilik sağlayacak şekilde takip edilmesidir. Kuş sınıfı ise tehdit oluşturmeyen unsurları temsil etmekte olup, modelin çevresel ayırım yeteneğini artırmak amacıyla eğitim sürecine dahil edilmiştir. Ancak hesaplama yükünü minimize etmek ve sistem kaynaklarını verimli kullanmak adına takip mekanizması yalnızca “drone” sınıfı için uygulanmıştır.

YOLO tabanlı nesne tespit modelleri, tekil görüntüler üzerinde yüksek doğrulukla sonuç üretse de, video akışı gibi zamansal veri içeren senaryolarda hedefin sürekliliğinin sağlanması doğrudan bu modellerin kapsamı dışında kalmaktadır [9]. Ardışık kareler arasında oluşabilecek ani konum değişimleri, tespit kayıpları veya düşük güven skorlu çıktılar, sistemin kararsız davranmasına neden olabilmektedir.

Bu tür problemlerin önüne geçebilmek amacıyla, tespit sonuçlarını zamansal olarak değerlendiren ve kısa süreli dalgalanmaları filtreleyen ek bir takip yaklaşımı geliştirilmiştir. Bu yapı sayesinde, tekil karelerde oluşabilecek hatalı veya geçici tespitlerin sistem davranışına doğrudan yansması engellenmiş ve daha kararlı bir takip performansı elde edilmiştir.

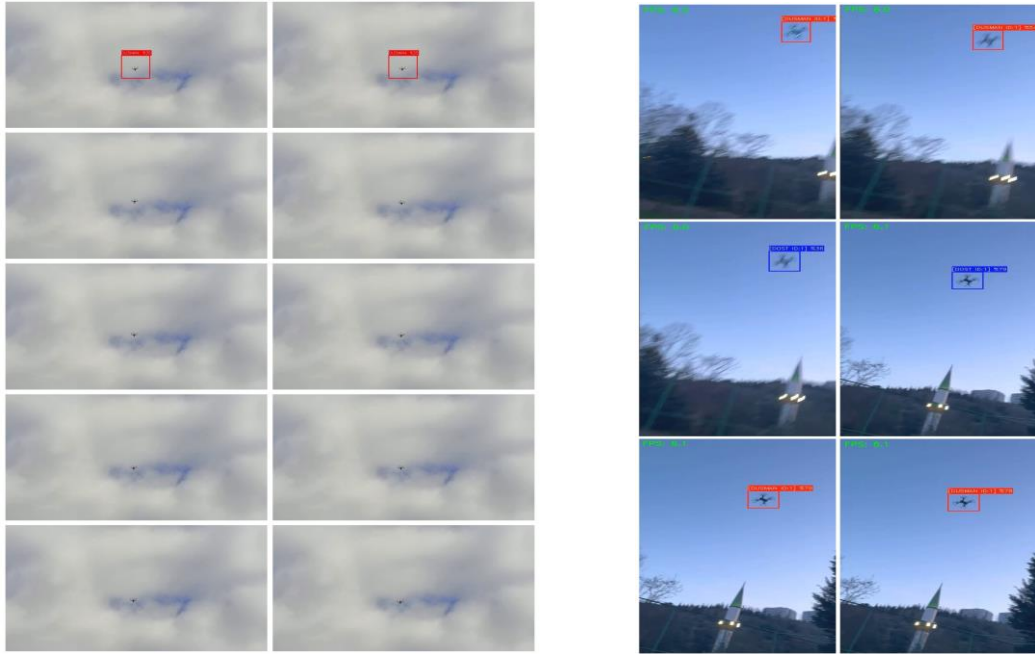
4.3.1 Küçük Nesnelerde Tespit Problemi

Geniş, tekdüze ve derinlik algısının sınırlı olduğu gökyüzü sahnelerinde, çok küçük piksel alanları kaplayan nesnelerin (small object detection) otonom olarak tespit edilmesi ve takip edilmesi, literatürde en zorlu görüntü işleme problemlerinden biri olarak değerlendirilmektedir. Özellikle yüksek irtifa ve uzak mesafelerde gözlemlenen drone ve kuş nesneleri, benzer görsel karakteristikler (küçük silüet yapısı, sınırlı detay, hareket benzerliği) sergilediğinden, bu durum nesne tespit modellerinin performansını doğrudan etkilemektedir.

Bu çalışma kapsamında yapılan gözlemler, YOLO tabanlı tespit modelinin tek başına kullanılması durumunda iki temel kararsızlık problemine yol açtığını göstermiştir:

1. Tespit Süreksizliği (Flickering / Bounding Box Kaybı):Parçalı bulut yapısı, ani aydınlatma değişimleri veya hedefin kısa süreli odak dışına çıkması gibi durumlarda model, nesneye ait öznitelikleri geçici olarak algılayamamakta ve tespit sınırlarını kaybedebilmektedir. Bu durum, gerçek zamanlı video akışında (örneğin 30 FPS) hedefin ekranda kesintili ve titreşimli (flickering) şekilde görünmesine neden olmaktadır.

2. Sınıf Kararsızlığı (Misclassification):Nesnenin kamera ile olan mesafesi arttıkça görüntü üzerindeki piksel temsili önemli ölçüde azalmaktadır. Bu koşullarda model, nesnenin “kuş” (dost) veya “drone” (düşman) sınıfına ait olduğunu ayırt edecek yeterli ayırt edici özelliğe ulaşamamaktadır. Sonuç olarak aynı hedef üzerinde ardışık kareler boyunca sınıf etiketinin “kuş–drone–kuş” şeklinde dalgalandığı gözlemlenmiştir. Bu kapsamda, tespit süreksizliği ve sınıf kararsızlığı problemlerinin etkileri örnek kareler üzerinden Şekil 4.2’de sunulmuştur.



Şekil 4.2: Tespit Sınırlarının Kaybolması ve Kararsız Tespitler

Şekil 4.2’de, küçük ölçekli hedeflerin tespiti sırasında karşılaşılan kararsızlık problemleri ardışık video kareleri üzerinden gösterilmektedir. Sol taraftaki örneklerde, hedef nesnenin belirli karelerde tespit edilemediği ve sınır kutularının

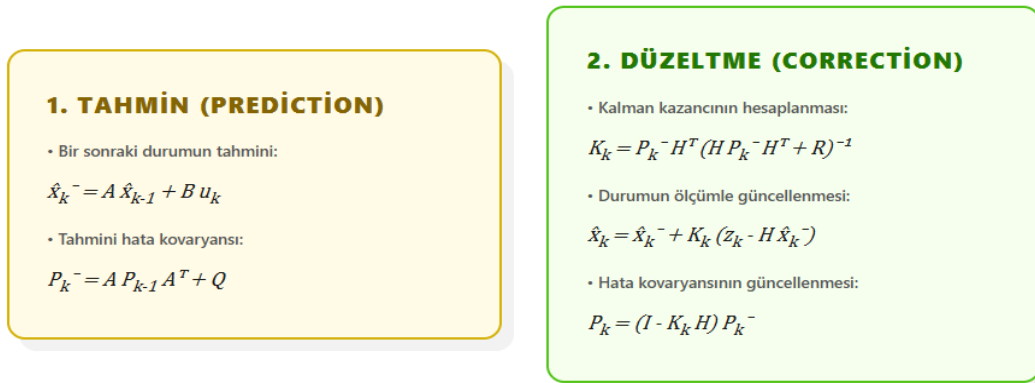
(bounding box) süreksiz şekilde kaybolup yeniden oluştuğu gözlemlenmektedir. Sağ taraftaki örneklerde ise aynı hedef üzerinde sınıf etiketinin ardışık kareler boyunca değişkenlik gösterdiği (kuş–drone–kuş) görülmektedir.

Bu durumlar, özellikle uzak mesafe, düşük piksel temsili ve çevresel faktörlerin etkisi altında, nesne tespit modellerinin kararlı sonuç üretmekte zorlandığını ortaya koymaktadır. Elde edilen bulgular, bu tür sistemlerde yalnızca tespit modeline dayalı bir yaklaşımın yeterli olmadığını ve zamansal tutarlılığı sağlayan ek algoritmaların gerekliliğini açıkça göstermektedir.

4.3.2 Kalman Filtresi ile YOLO Entegrasyonu

Küçük nesnelerde gözlemlenen tespit süreksizliği (flickering) problemini azaltmak ve hedef takibinde sürekliliği sağlamak amacıyla, YOLO tabanlı tespit çıktıları üzerine durum tahmini (state estimation) temelli bir yaklaşım olarak Kalman filtresi entegre edilmiştir [8].

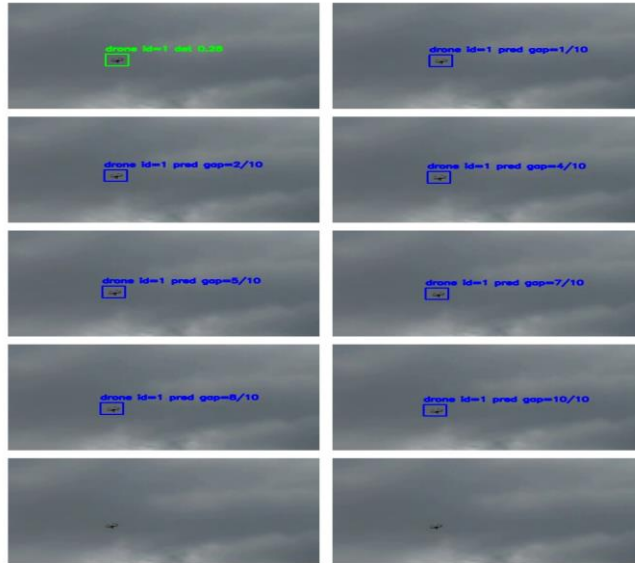
Kalman filtresi, gürültülü ölçüm verilerinden sistemin anlık durumunu ve gelecekteki konumunu tahmin etmeye olanak tanıyan özyinelemeli (recursive) bir algoritmadır [8]. Algoritma, temel olarak iki aşamadan oluşmaktadır: tahmin (prediction) ve güncelleme (update). Tahmin aşamasında hedefin mevcut konum ve hız bilgileri kullanılarak bir sonraki konumu öngörülürken, güncelleme aşamasında model çıktısı ile bu tahmin düzeltilerek hata minimize edilmektedir. Kalman filtresinin tahmin ve güncelleme aşamalarından oluşan temel çalışma yapısı Şekil 4.3'te gösterilmektedir.



Şekil 4.3: Kalman Filtresinin Yapısı [8]

Bu çalışma kapsamında Kalman filtresi, YOLO modelinin hedefi geçici olarak tespit edemediği karelerde devreye girerek hedefin son bilinen konum ve hareket eğiliminden yararlanarak tahmini bir konum üretmek amacıyla kullanılmıştır. YOLO tarafından başarılı şekilde tespit edilen karelerde elde edilen bounding box koordinatları filtreye ölçüm girdisi olarak sağlanırken, tespit kaybı yaşanan durumlarda Kalman filtresi tarafından üretilen tahmini konum, hedefin sürekliliğini korumak amacıyla sistem çıktısına yansıtılmıştır.

Ancak yapılan deneysel değerlendirmeler, drone hedeflerinin yüksek manevra kabiliyeti ve ani hız değişimleri sergilemesi nedeniyle sabit hız varsayımına dayanan Kalman modelinin bazı durumlarda yetersiz kaldığını göstermiştir. Özellikle ani yön değişimlerinde tahmin edilen konumun gerçek hedef konumundan sapma (drift) eğilimi gösterdiği gözlemlenmiştir. Kalman filtresi ile desteklenen YOLO tabanlı takip yaklaşımının çıktıları örnek kareler üzerinden Şekil 4.4'te sunulmuştur.

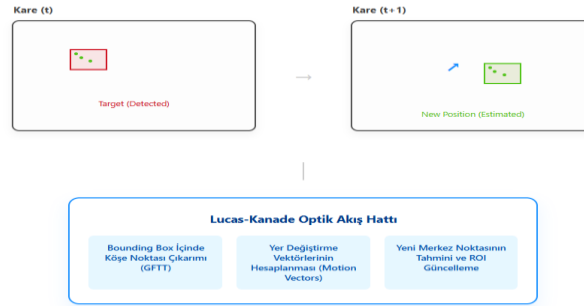


Şekil 4.4: YOLO+Kalman Filtresi Yaklaşımının Çıktıları

Bu bulgular doğrultusunda, Kalman filtresinin tek başına kararlı bir takip çözümü sunmakta sınırlı kaldığı ve daha esnek, doğrudan görüntü verisine dayalı yaklaşımlarla desteklenmesi gerektiği sonucuna varılmıştır.

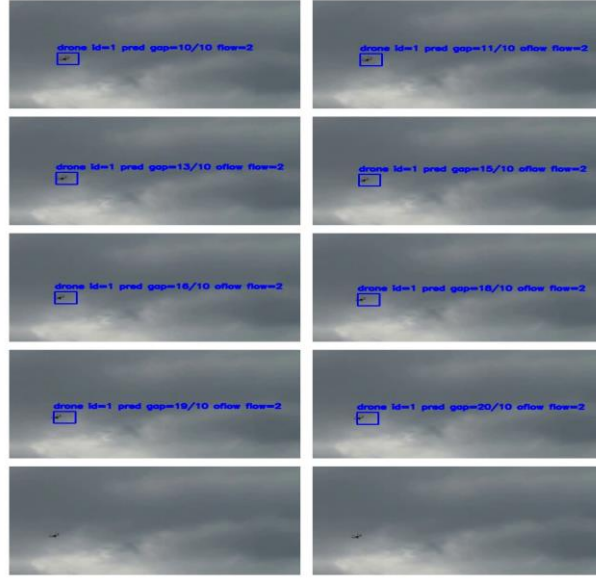
4.3.3 Optik Akış (Optical Flow) ile YOLO Entegrasyonu

Optik akış (Optical Flow), ardışık video kareleri arasındaki piksel hareketlerini vektörel olarak hesaplayan bir bilgisayarlı görü yöntemidir. Bu yaklaşım, görüntüdeki belirli öznitelik noktalarının (feature points) kareler arasındaki yer değişimini izleyerek nesnenin hareket yönü ve hızının doğrudan görüntü verisinden çıkarılmasına olanak tanımaktadır [6]. Lucas–Kanade ve Farnebäck gibi yöntemler, bu amaçla yaygın olarak kullanılan optik akış algoritmaları arasında yer almaktadır [6]. Optik akış yönteminin kareler arası hareket tahminine dayalı temel çalışma prensibi Şekil 4.5’te gösterilmektedir.



Şekil 4.5: Optical Flow Akış Diyagramı [6]

Bu çalışmada, Kalman filtresinin ani hareketlere karşı sınırlı performans göstermesi üzerine, hedef takibinde ek bir katman olarak optik akış tabanlı yaklaşım değerlendirilmiştir. Bu kapsamda, YOLO modeli tarafından tespit edilen hedefin sınır kutusu içerisindeki öznitelik noktaları çıkarılarak, izleyen karelerde bu noktaların konum değişimleri Lucas–Kanade yöntemi ile hesaplanmıştır. Elde edilen hareket vektörleri kullanılarak hedefin yeni konumu güncellenmiş ve bounding box merkezi bu doğrultuda yeniden belirlenmiştir. YOLO, Kalman filtresi ve optik akış yöntemlerinin birlikte kullanıldığı hibrit takip yaklaşımına ait örnek çıktılar Şekil 4.6’da sunulmuştur.



Şekil 4.6: YOLO+Kalman+Optical Flow Yaklaşımının Çıktıları

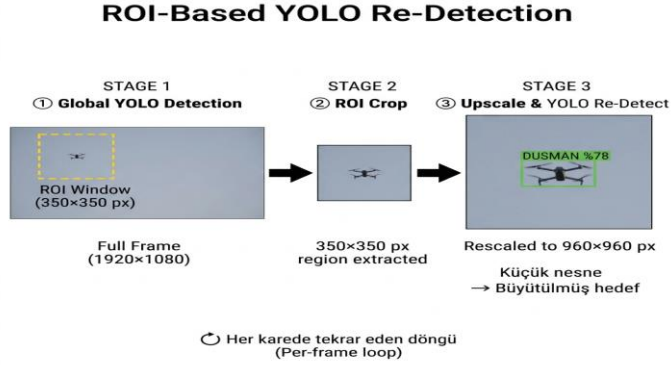
Ancak gerçekleştirilen testler, küçük ölçekli hedeflerde optik akış yönteminin sınırlı performans sergilediğini ortaya koymuştur. Drone nesnelerinin görüntü üzerindeki piksel temsili oldukça düşük olduğundan, güvenilir ve yeterli sayıda öznitelik noktası çıkarılamamaktadır. Buna ek olarak, arka planda bulunan bulut hareketleri gibi dinamik unsurların oluşturduğu gürültü (noise), optik akış vektörlerinin doğruluğunu olumsuz yönde etkilemiştir.

Ayrıca optik akış hesaplamalarının getirdiği ek işlem yükü, sistemin kare hızında (FPS) düşüşe neden olmuş ve gömülü sistem performansı açısından dezavantaj oluşturmuştur. Bu bulgular doğrultusunda, optik akış tabanlı yaklaşımın küçük nesne takibi ve gerçek zamanlı uygulamalar için tek başına yeterli bir çözüm sunmadığı değerlendirilmiştir.

4.3.4 İlgi Alanı (ROI) Tabanlı Yeniden Tespit Yaklaşımı

İlgi Alanı (Region of Interest - ROI), görüntü işleme uygulamalarında tüm görüntü yerine yalnızca anlamlı bilginin bulunduğu alt bölgenin seçilerek işlenmesini sağlayan bir yöntemdir. Bu yaklaşım, gereksiz arka plan piksellerinin işlenmesini engelleyerek hem hesaplama maliyetini düşürmekte hem de modelin odaklanma yeteneğini artırmaktadır [6]. Özellikle küçük ölçekli ve belirli bir konum sürekliliği

sergileyen nesnelerin takibinde, önceki kareye ait konum bilgisinden yararlanılarak oluşturulan dinamik ROI bölgeleri tespit performansını önemli ölçüde iyileştirmektedir. ROI tabanlı yeniden tespit yaklaşımının çalışma adımları ve veri akışı Şekil 4.7’de gösterilmektedir.



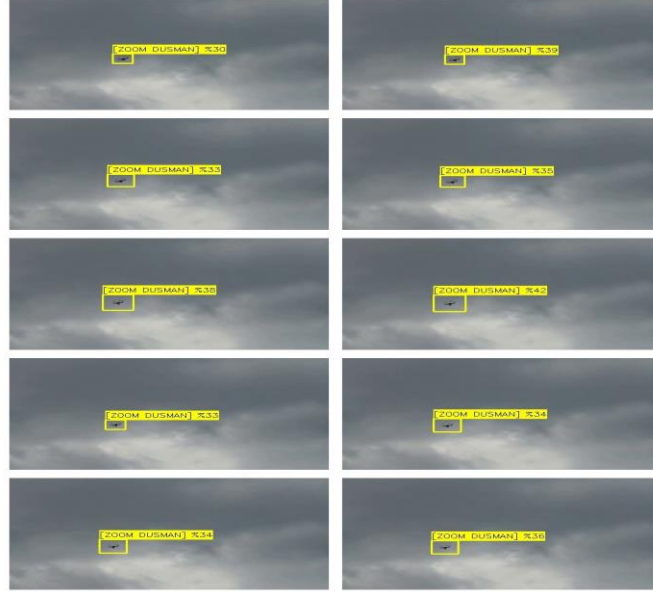
Şekil 4.7: Kullanılan ROI Yapısını Gösteren Diyagram

Bu çalışmada, Kalman filtresi ve optik akış tabanlı yaklaşımların gürültüye duyarlılığı ve işlem maliyeti açısından sınırlı kalması üzerine, tamamen görüntü verisine dayalı (vision-only) bir ROI tabanlı yeniden tespit yaklaşımı geliştirilmiştir. Önerilen yöntemde, YOLO modeli hedefi global arama ile ilk kez tespit ettiğinde, hedefin son bilinen konumu referans alınarak etrafında genişletilmiş bir ROI penceresi tanımlanmaktadır.

İzleyen karelerde model, tüm görüntü yerine yalnızca bu ROI bölgesi üzerinde çalıştırılmaktadır. ROI bölgesi model giriş boyutuna (960×960) yeniden ölçeklendirilerek işlenmekte ve bu sayede küçük piksel alanı kaplayan hedefler model perspektifinde büyütülmüş bir görünüme kavuşmaktadır. Bu "optik yakınlaştırma (zoom)" etkisi, küçük nesne tespit performansını doğrudan artırmaktadır.

Önerilen yaklaşım, hem hesaplama yükünü azaltarak sistemin kare hızını (FPS) korumakta hem de hedefin sürekliliğini artırarak daha kararlı bir takip performansı sunmaktadır. Ekran üzerinde yer alan [ZOOM DUSMAN] etiketi, sistemin aktif olarak ROI tabanlı takip modunda çalıştığını göstermektedir. YOLO ve ROI tabanlı

takip yaklaşımının küçük nesneler üzerindeki performansını gösteren örnek çıktılar Şekil 4.8’de sunulmuştur.



Şekil 4.8: YOLO+ROI Yaklaşımının Çıktıları

Geliştirilen bu yöntem, önceki yaklaşımlarla karşılaştırmalı olarak değerlendirilmiş ve elde edilen sonuçlar Tablo 4.3’de özetlenmiştir.

Tablo 4.3: Kalman, Optical Flow ve ROI Yaklaşımlarının Karşılaştırılması

Kriter	YOLO+Kalman	YOLO+Kalman+Optik Akış	YOLO+ROI
Fizik/Model Bağımlılığı	Yüksek (sabit hız varsayımı)	Orta	Yok (Saf Görüntü)
Manevra Dayanıklılığı	Düşük	Orta	Yüksek
Arka Plan Gürültüsüne Duyarlılık	Yüksek	Çok Yüksek	Düşük
Küçük Nesne Tespit Başarımı	Orta	Orta	Yüksek (Zoom Etkisi)
İşlemci Maliyet (FPS Etkisi)	Orta	Yüksek	Düşük
Sistem Karmaşıklığı	Orta	Yüksek	Düşük

Tablo 4.3’de sunulan karşılaştırma sonuçları doğrultusunda, YOLO + ROI yaklaşımının küçük nesne tespiti ve gerçek zamanlı performans açısından diğer yöntemlere göre belirgin üstünlükler sağladığı görülmüştür. Bu nedenle, sistemin nihai hedef takip mimarisi YOLO + ROI temeli üzerine kurulmuştur. Bununla birlikte, dinamik sahnelerde tespit kararlılığını daha da artırmak amacıyla, zamansal tutarlılığı sağlayan ek bir yazılımsal katman olarak zamansal oy çoğunluğu (temporal majority voting) mekanizması sisteme entegre edilmiştir.

4.3.5 Zamansal Oy Çoğunluğu ile Kararlı Hedef Sınıflandırması

Oy çoğunluğu (majority voting), birden fazla bağımsız kararın bir araya getirilerek en sık tekrar eden sonucun nihai karar olarak seçilmesine dayanan bir topluluk öğrenmesi (ensemble learning) yaklaşımıdır [9]. Bu yöntem, özellikle tekil tahminlerin gürültüye duyarlı olduğu durumlarda, karar süreçlerini kararlı hale getirmek amacıyla yaygın olarak kullanılmaktadır. Görüntü tabanlı sistemlerde ise bu yaklaşım, ardışık video karelerinden elde edilen model çıktılarının zamansal bir pencere içerisinde değerlendirilmesiyle uygulanmaktadır.

YOLO + ROI tabanlı takip mimarisi, hedef konumunun kararlı bir şekilde izlenmesini sağlasa da, uzak mesafede bulunan küçük hedeflerde modelin sınıflandırma çıktısının tek karelik piksel belirsizliklerine karşı hassas olduğu gözlemlenmiştir. Bu durum, aynı hedef üzerinde ardışık kareler boyunca “drone” ve “kuş” etiketleri arasında hızlı geçişlere neden olmakta ve sistem güvenilirliğini olumsuz etkilemektedir.

Bu problemi gidermek amacıyla, her bir hedef için son N kareye ait sınıf tahminleri bir zaman tamponu (temporal buffer) içerisinde saklanmıştır. Nihai sınıf etiketi, bu pencere içerisindeki çoğunluk kararına göre belirlenmektedir. Bu sayede, tek kareye ait hatalı veya gürültü kaynaklı sınıflandırma sonuçlarının sistem çıktısına doğrudan yansması engellenmiştir.

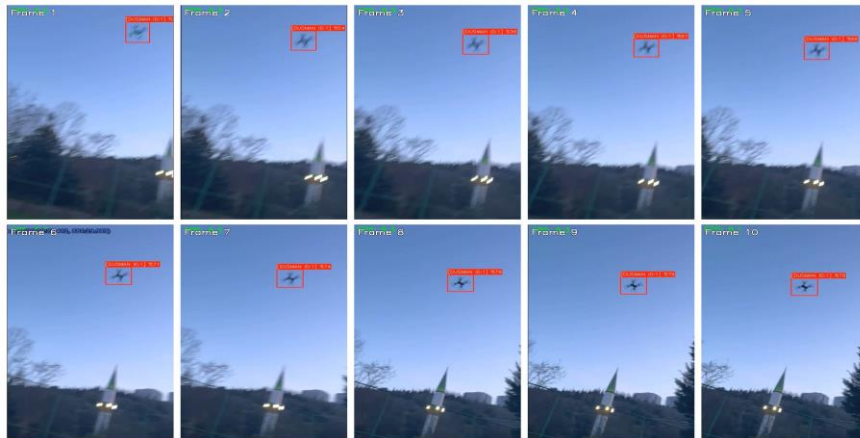
Uygulama kapsamında pencere boyutu $N = 7$ olarak belirlenmiş olup, sınıf değişiminin gerçekleşebilmesi için en az 4 karede aynı sınıfın tespit edilmesi koşulu

getirilmiştir. Bu yaklaşım sayesinde sistemin sınıflandırma kararlılığı önemli ölçüde artırılmış ve hedef üzerindeki “düşman” kilidinin kısa süreli belirsizliklerden etkilenmesi engellenmiştir. Zamansal sınıf dalgalanmalarını ve kararsız tespit örüntülerini gösteren örnek çıktılar Şekil 4.9’da sunulmuştur.



Şekil 4.9: Kararsız Tespitlerin Örnek Çıktıları

Şekil 4.9’da, ardışık video kareleri boyunca aynı hedef üzerinde uygulanan zamansal oy çoğunluğu mekanizmasının etkisi gösterilmektedir. Tekil karelerde model çıktısının sınıf belirsizliği göstermesine rağmen, zaman tamponu ($N=7$) üzerinden yapılan değerlendirme sonucunda hedefin “düşman” sınıf etiketi kararlı şekilde korunmaktadır. Bu durum, kısa süreli yanlış sınıflandırmaların sistem çıktısına doğrudan yansımaları engelleyerek takip güvenilirliğini artırmaktadır. Zamansal oy çoğunluğu mekanizmasının uygulanması sonucunda elde edilen kararlı tespit örnekleri Şekil 4.10’da sunulmuştur.



Şekil 4.10: Doğru Kararlı Tespitlerin Örnek Çıktıları

4.3.6 Çoklu Hedef Takibi, ID Ataması ve HUD Arayüzü

Gerçek dünya senaryolarında kamera görüş alanına eşzamanlı olarak birden fazla hedef nesnenin (drone) girebilmesi, sistemin yalnızca tek bir hedefi izlemekle sınırlı kalmamasını gerektirmektedir. Bu doğrultuda, sahnedeki tüm hedeflerin bağımsız olarak izlenebilmesi amacıyla sisteme bir Çoklu Nesne Takibi (Multi-Object Tracking - MOT) katmanı entegre edilmiştir .

Önerilen yaklaşımda, kamera görüş alanına giren her yeni hedef nesne ilk tespit anında benzersiz bir kimlik numarası (ID) ile etiketlenmektedir. Ardışık video karelerinde bu kimlikler, nesneler arasındaki örtüşme oranını ölçen Intersection over Union (IoU) metriği kullanılarak yeniden eşleştirilmektedir. Tespit kaybı yaşanan kısa süreli durumlarda ise, hedefe ait kimlik bilgisi belirli bir zaman aralığı boyunca sistem belleğinde tutulmakta (ID retention) ve böylece takip sürekliliği korunmaktadır. Bu mekanizma sayesinde sahnedeki her hedef, TRK [ID:1], TRK [ID:2] gibi ayrı kimlikler altında kesintisiz olarak izlenebilmektedir.

Çoklu hedef takibi gerçekleştirilirken, sistemin operasyonel olarak tek bir hedefe odaklanması gerekmektedir. Bu amaçla, hedef önceliklendirme kriteri olarak ekran merkezine olan **Öklid uzaklığı (Euclidean distance to frame center)** kullanılmıştır. Her video karesinde, aktif takipte bulunan tüm hedeflerin merkez koordinatları ile ekranın geometrik merkezi arasındaki mesafe hesaplanmakta ve en düşük uzaklığa sahip hedef otomatik olarak öncelikli hedef olarak seçilmektedir [6]. Nişangah (LCK – Locked) bu hedef üzerine yönlendirilerek aktif takip durumu sağlanmaktadır.

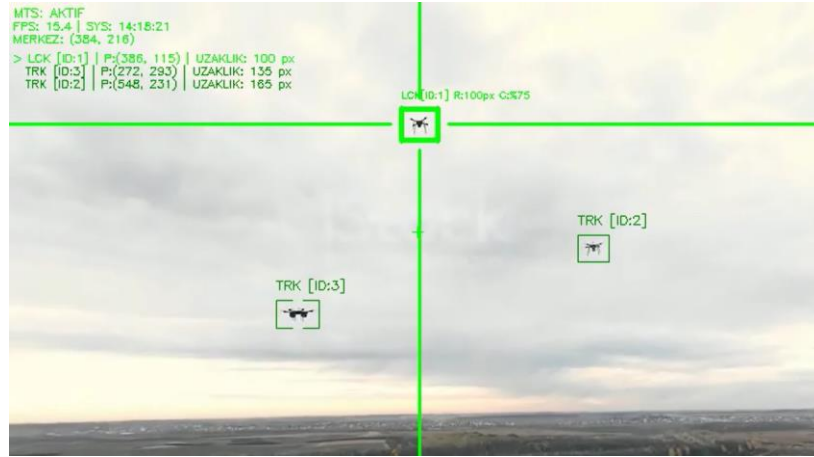
Kilitli hedefin görüş alanından çıkması veya tespit edilememesi durumunda sistem, öncelik sırasına göre bir sonraki en yakın hedefe otomatik olarak geçiş yapmaktadır. Bu sayede nişangahın sürekli aktif kalması sağlanmakta ve hedef kaybı durumlarında sistemin tepki süresi minimize edilmektedir.

Sistemin operasyonel durumunu kullanıcıya anlık olarak aktarabilmek amacıyla, gerçek zamanlı video akışı üzerine entegre edilmiş bir **Head-Up Display (HUD)** arayüzü geliştirilmiştir. Bu arayüz, hedef kimlikleri, kilit durumu ve takip bilgilerini

yeşil monokrom bir gösterim ile sunarak, kullanıcıya askeri sistemlere benzer bir görsel deneyim sağlamaktadır. Geliştirilen HUD arayüzünün gerçek zamanlı hedef işaretleme performansını gösteren örnek çıktı Şekil 4.11 ve Şekil 4.12’de sunulmuştur.



Şekil 4.11: Düşman(Drone) Hedefinin Hud Arayüzü ile İşaretlenmesi 1



Şekil 4.12: Düşman(Drone) Hedefinin Hud Arayüzü ile İşaretlenmesi 2

4.4 Raspberry Pi 5 Gömülü Platform Üzerinde Sistemin İlk Uygulaması

Bölüm 3.3 ve 3.4’te gerçekleştirilen model eğitimi ve algoritma geliştirme süreçlerinin ardından, önerilen sistemin gerçek dünya koşullarındaki performansının değerlendirilebilmesi amacıyla gömülü bir platform üzerinde uygulanması gerekmektedir. Bu doğrultuda, sınırlı hesaplama kaynaklarına sahip bir uç cihaz olarak Raspberry Pi 5 platformu tercih edilmiştir [3].

Bu bölümde, eğitilen YOLO modelinin gömülü sisteme entegrasyonu, model dönüşüm süreçleri ve gerçek zamanlı performans analizleri ele alınmaktadır.

Özellikle modelin gömülü platformda çalışabilirliğini sağlamak amacıyla yapılan optimizasyon adımları, donanım kısıtları altında elde edilen kare hızı (FPS) değerleri ve sistemin genel çalışma davranışı detaylı olarak incelenmiştir.

Ayrıca, geliştirilen hedef tespit ve takip algoritmalarının Raspberry Pi 5 üzerinde gerçek zamanlı video akışı ile entegrasyonu gerçekleştirilmiş ve sistemin saha koşullarına yakın senaryolarda doğrulama testleri yapılmıştır.

4.4.1 İşletim Sisteminin Kurulumu

Raspberry Pi 5 platformu, farklı kullanım senaryolarına yönelik çeşitli işletim sistemlerini desteklemektedir. Bu sistemlerin başlatma süresi, bellek kullanımı, işlemci yükü ve donanım uyumluluğu gibi temel performans kriterleri Tablo 4.4’de karşılaştırmalı olarak sunulmaktadır.

Tablo 4.4: Raspberry Pi 5 için Uygun İşletim Sistemleri ve Özellikleri

İşletim Sistemi	Boot Süresi	RAM Kullanımı	CPU Yükü	Güncellik	Depolama Kullanımı	Donanım Uyumu
Raspberry Pi OS	~17 sn	~200 MB	Orta	Yüksek	~2.4 GB	Tam uyumlu
Ubuntu Server	~22 sn	~300 MB	Yüksek	Yüksek	~3.0 GB	İyi
DietPi	~12 sn	~80 MB	Düşük	Orta	~1.2 GB	Yüksek
LineageOS (A13)	~35 sn	~600 MB	Orta-Yüksek	Orta	~5.0 GB	Orta
LibreELEC	~15 sn	~250 MB	Düşük	Orta	~2.0 GB	Yüksek
Buildroot/Yocto	10–30 sn	50–100 MB	Düşük	Geliştiriciye bağlı	~800 MB+	Özelleştirilebilir

Tablo 4.4 incelendiğinde, hafif dağıtımların düşük kaynak tüketimi avantajı sunduğu, ancak donanım sürücüler ve yazılım ekosistemi açısından bazı kısıtlar barındırdığı görülmektedir. Buna karşılık, daha kapsamlı işletim sistemleri ise yüksek uyumluluk ve geniş kütüphane desteği sağlamakla birlikte, gömülü sistemler için kritik olan kaynak kullanımını artırabilmektedir.

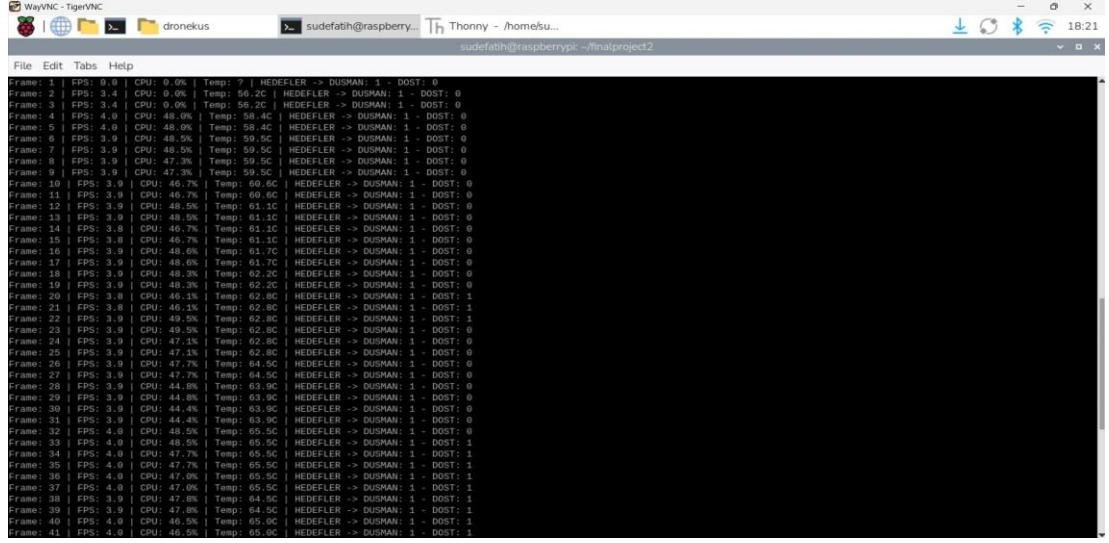
Bu değerlendirmeler doğrultusunda, proje kapsamında Raspberry Pi OS (64-bit) işletim sistemi tercih edilmiştir. Tercih edilmesinin temel nedenleri; Raspberry Pi Foundation tarafından resmi olarak geliştirilmesi [3], donanıma özgü sürücü ve kütüphanelerle (kamera altyapısı, GPIO arayüzleri ve NCNN gibi derin öğrenme kütüphaneleri) yüksek uyumluluk sağlaması ve geniş kullanıcı topluluğu sayesinde geliştirme sürecinde destek kolaylığı sunmasıdır.

Sistem kurulumu sonrasında, cihazın fiziksel çevre birimlerine ihtiyaç duymadan yönetilebilmesi amacıyla uzaktan erişim altyapısı yapılandırılmıştır. Bu kapsamda, WayVNC ve TigerVNC tabanlı bir yapı kullanılarak VNC protokolü üzerinden erişim sağlanmış ve geliştirme ile test süreçleri uzaktan bağlantı aracılığıyla yürütülmüştür.

4.4.2 YOLO Modelinin PyTorch Formatında Testi

Raspberry Pi 5 üzerinde gerçekleştirilen ilk performans değerlendirmesinde, Google Colab ortamında eğitilmiş ve .pt uzantılı PyTorch formatındaki YOLOv8n modeli doğrudan çalıştırılmıştır. Bu test, NCNN tabanlı optimizasyon sürecine geçilmeden önce modelin ham performansını ölçmek ve karşılaştırma amacıyla bir referans (baseline) oluşturmak üzere tasarlanmıştır.

Test sürecinde görüntü kaynağı olarak önceden kaydedilmiş bir video kullanılmış; sistem her kare için kare hızı (FPS), CPU kullanım oranı, işlemci sıcaklığı ve tespit edilen hedef sayısı (düşman/dost) gibi metrikleri terminal çıktısı olarak raporlamıştır. Elde edilen örnek çıktılar Şekil 4.13'te sunulmaktadır.



```
File Edit Tabs Help
Frame: 1 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 2 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 3 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 4 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 5 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 6 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 7 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 8 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 9 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 10 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 11 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 12 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 13 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 14 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 15 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 16 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 17 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 18 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 19 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 20 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 21 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 22 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 23 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 24 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 25 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 26 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 27 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 28 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 29 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 30 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 31 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 32 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 33 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 34 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 35 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 36 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 37 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 38 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 39 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 40 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
Frame: 41 | FPS: 3.9 | CPU: 48.0% | Temp: 56.2C | HEDEFLEK -> DUSMAN: 1 - DOST: 0
```

Şekil 4.13:Pytorch Modelin Performansının Terminal Çıktısı

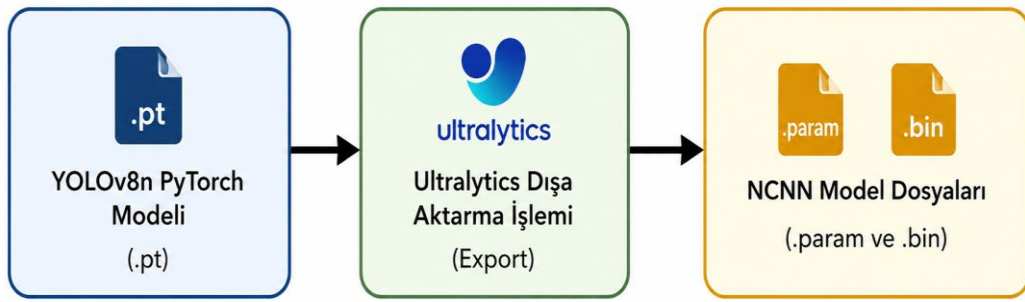
Şekil 4.13 incelendiğinde, sistemin ortalama yaklaşık 3.9 FPS seviyesinde kararlı bir performans sergilediği görülmektedir. CPU kullanımının %44–49 aralığında seyrettiği, işlemci sıcaklığının ise test süresince 56.2°C’den yaklaşık 65°C seviyesine yükseldiği gözlemlenmiştir. Bu sıcaklık artışı ve işlem yükü, PyTorch çalışma ortamının ARM tabanlı işlemcilerde herhangi bir donanım hızlandırması (GPU/NPU) olmaksızın tamamen CPU üzerinde çalışmasından kaynaklanmaktadır. Elde edilen bu performans değerleri, gerçek zamanlı video işleme sistemleri için genel kabul gören 24–30 FPS aralığının oldukça altında kalmaktadır. Bu durum, PyTorch formatındaki modelin Raspberry Pi 5 gibi sınırlı donanım kaynaklarına sahip gömülü platformlarda doğrudan üretim ortamında kullanılmasının uygun olmadığını açıkça göstermektedir.

4.4.3 YOLO Modelinin NCNN Formatına Dönüşümü ve Testi

NCNN (Neural Network Compute Kit), Tencent tarafından geliştirilen ve özellikle ARM tabanlı mobil ve gömülü sistemler için optimize edilmiş açık kaynaklı bir derin öğrenme çıkarım (inference) çerçevesidir [5]. NCNN, derin öğrenme modellerini platforma özgü derlenmiş ikili formatlara (.param ve .bin) dönüştürerek ARM NEON SIMD komut setini etkin biçimde kullanmakta ve bu sayede genel amaçlı çalışma

zamanlarının (runtime) oluşturduğu ek işlem yükünü (overhead) minimize etmektedir [1].

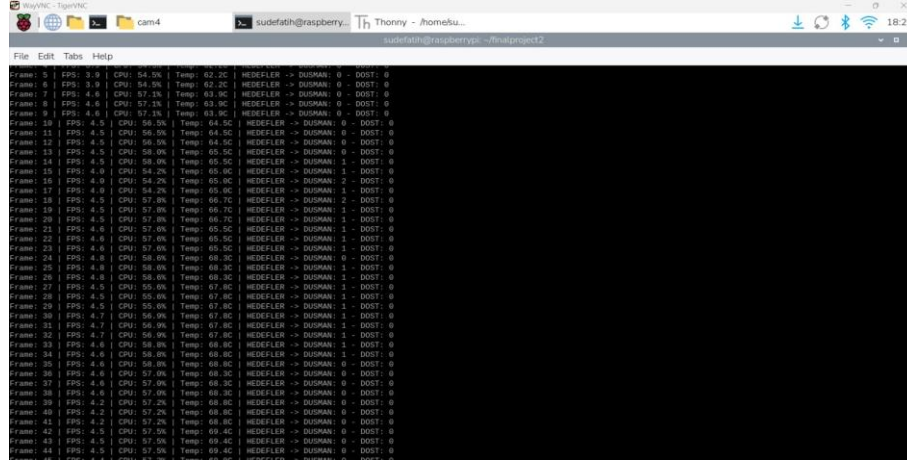
Bu çalışma kapsamında, PyTorch .pt formatında eğitilmiş YOLOv8n modeli, gömülü sistem üzerinde daha verimli çalışabilmesi amacıyla NCNN formatına dönüştürülmüştür. Dönüşüm işlemi, Ultralytics kütüphanesinin yerleşik dışa aktarma (export) fonksiyonu kullanılarak gerçekleştirilmiştir [1] YOLOv8n modelinin NCNN formatına dönüştürülmesi süreci Şekil 4.14'te gösterilmiştir.



Şekil 4.14:YOLOv8n modelinin PyTorch formatından NCNN formatına dönüştürülmesi

Bu işlem sırasında model öncelikle ONNX formatına çevrilmekte, ardından NCNN araç zinciri kullanılarak ağ yapısını tanımlayan .param ve ağırlıkları içeren .bin dosyaları oluşturulmaktadır. Elde edilen model dosyaları Raspberry Pi 5 platformuna aktarılmış ve performans testleri bu ortamda gerçekleştirilmiştir.

NCNN dönüşümünün ardından, PyTorch modeli ile gerçekleştirilen test senaryosu aynen korunarak, aynı video girdisi ve performans metrikleri (FPS, CPU kullanımı ve işlemci sıcaklığı) üzerinden yeniden ölçüm yapılmıştır. Elde edilen sonuçlar Şekil 4.15'te sunulmaktadır.



```
Frame: 5 | FPS: 4.8 | CPU: 54.0% | Temp: 62.20 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
Frame: 6 | FPS: 4.8 | CPU: 54.0% | Temp: 62.20 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
Frame: 7 | FPS: 4.8 | CPU: 57.0% | Temp: 63.80 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
Frame: 8 | FPS: 4.6 | CPU: 57.1% | Temp: 63.80 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
Frame: 9 | FPS: 4.8 | CPU: 57.1% | Temp: 63.80 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
Frame: 10 | FPS: 4.5 | CPU: 58.0% | Temp: 64.50 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
Frame: 11 | FPS: 4.5 | CPU: 58.0% | Temp: 64.50 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
Frame: 12 | FPS: 4.5 | CPU: 58.0% | Temp: 64.50 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
Frame: 13 | FPS: 4.5 | CPU: 58.0% | Temp: 65.50 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
Frame: 14 | FPS: 4.5 | CPU: 58.0% | Temp: 65.50 | HEDEFLEX -> DUSMAN: 1 - DOST: 0
Frame: 15 | FPS: 4.9 | CPU: 54.2% | Temp: 65.80 | HEDEFLEX -> DUSMAN: 1 - DOST: 0
Frame: 16 | FPS: 4.0 | CPU: 54.2% | Temp: 65.90 | HEDEFLEX -> DUSMAN: 2 - DOST: 0
Frame: 17 | FPS: 4.8 | CPU: 58.2% | Temp: 65.80 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
Frame: 18 | FPS: 4.5 | CPU: 57.8% | Temp: 66.70 | HEDEFLEX -> DUSMAN: 2 - DOST: 0
Frame: 19 | FPS: 4.5 | CPU: 57.8% | Temp: 66.70 | HEDEFLEX -> DUSMAN: 1 - DOST: 0
Frame: 20 | FPS: 4.5 | CPU: 57.8% | Temp: 66.70 | HEDEFLEX -> DUSMAN: 1 - DOST: 0
Frame: 21 | FPS: 4.6 | CPU: 57.0% | Temp: 65.50 | HEDEFLEX -> DUSMAN: 1 - DOST: 0
Frame: 22 | FPS: 4.8 | CPU: 57.0% | Temp: 65.50 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
Frame: 23 | FPS: 4.8 | CPU: 57.0% | Temp: 65.50 | HEDEFLEX -> DUSMAN: 1 - DOST: 0
Frame: 24 | FPS: 4.8 | CPU: 58.0% | Temp: 68.30 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
Frame: 25 | FPS: 4.8 | CPU: 58.0% | Temp: 68.30 | HEDEFLEX -> DUSMAN: 1 - DOST: 0
Frame: 26 | FPS: 4.8 | CPU: 58.0% | Temp: 68.30 | HEDEFLEX -> DUSMAN: 1 - DOST: 0
Frame: 27 | FPS: 4.5 | CPU: 55.0% | Temp: 67.80 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
Frame: 28 | FPS: 4.5 | CPU: 55.0% | Temp: 67.80 | HEDEFLEX -> DUSMAN: 1 - DOST: 0
Frame: 29 | FPS: 4.5 | CPU: 55.0% | Temp: 67.80 | HEDEFLEX -> DUSMAN: 1 - DOST: 0
Frame: 30 | FPS: 4.7 | CPU: 56.0% | Temp: 67.80 | HEDEFLEX -> DUSMAN: 1 - DOST: 0
Frame: 31 | FPS: 4.7 | CPU: 56.0% | Temp: 67.80 | HEDEFLEX -> DUSMAN: 1 - DOST: 0
Frame: 32 | FPS: 4.7 | CPU: 58.0% | Temp: 68.80 | HEDEFLEX -> DUSMAN: 1 - DOST: 0
Frame: 33 | FPS: 4.8 | CPU: 58.0% | Temp: 68.80 | HEDEFLEX -> DUSMAN: 1 - DOST: 0
Frame: 34 | FPS: 4.6 | CPU: 58.0% | Temp: 68.80 | HEDEFLEX -> DUSMAN: 1 - DOST: 0
Frame: 35 | FPS: 4.6 | CPU: 58.0% | Temp: 68.80 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
Frame: 36 | FPS: 4.8 | CPU: 57.0% | Temp: 68.30 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
Frame: 37 | FPS: 4.8 | CPU: 57.0% | Temp: 68.30 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
Frame: 38 | FPS: 4.5 | CPU: 57.0% | Temp: 68.30 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
Frame: 39 | FPS: 4.2 | CPU: 57.2% | Temp: 68.80 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
Frame: 40 | FPS: 4.2 | CPU: 57.2% | Temp: 68.80 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
Frame: 41 | FPS: 4.2 | CPU: 57.2% | Temp: 68.80 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
Frame: 42 | FPS: 4.5 | CPU: 57.0% | Temp: 69.40 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
Frame: 43 | FPS: 4.5 | CPU: 57.0% | Temp: 69.40 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
Frame: 44 | FPS: 4.5 | CPU: 57.0% | Temp: 69.40 | HEDEFLEX -> DUSMAN: 0 - DOST: 0
```

Şekil 4.15: NCNN Modelim Performansının Terminal Çıktısı

Test sonuçları incelendiğinde, NCNN formatının PyTorch'a kıyasla belirgin bir performans iyileşmesi sağladığı görülmektedir. Sistem, yaklaşık 4.0–4.8 FPS aralığında daha stabil bir çalışma sergilemiştir. CPU kullanım oranının %54–58 seviyelerine yükseldiği ve işlemci sıcaklığının 62°C ile 69°C arasında değiştiği gözlemlenmiştir. Bu durum, NCNN'in Raspberry Pi 5 üzerindeki ARM NEON mimarisini ve çok çekirdekli işlem yapısını daha etkin kullandığını göstermektedir. Elde edilen bulgular, NCNN dönüşümünün modelin gömülü platform üzerindeki verimliliğini önemli ölçüde artırdığını ortaya koymaktadır. Her ne kadar elde edilen kare hızları tam anlamıyla gerçek zamanlı eşik değerine ulaşmamış olsa da, PyTorch formatına kıyasla sağlanan performans artışı, sistemin gömülü ortamda uygulanabilirliğini mümkün kılan kritik bir iyileştirme olarak değerlendirilmektedir.

4.4.4 YOLOv8n Modelinin Farklı Çözünürlükteki Görüntü Kaynakları ile Karşılaştırılması

Gömülü platformlarda gerçek zamanlı performansı etkileyen kritik parametrelerden biri, işlenen ham video kaynağının çözünürlüğüdür. YOLO modeli sabit bir giriş boyutunda (örneğin 640×640) çalışıyor olsa da, her video karesi modele verilmeden önce görüntünün okunması, belleğe alınması ve yeniden ölçeklendirilmesi (resize) gerekmektedir. Bu ön işleme adımları, özellikle yüksek çözünürlüklü görüntülerde önemli bir hesaplama yükü (overhead) oluşturmaktadır. Modelin Raspberry Pi 5

üzerinde gerçekleştirilen çıkarım sürecine ait performans metrikleri (FPS, CPU kullanımı ve sıcaklık değerleri) Şekil 4.16’da gösterilmektedir.

Frame	FPS	CPU	Temp	HEDEFER	DUSMAN	DOST
Frame 58	11.2	83.4%	67.8C	HEDEFER	DUSMAN	1 - DOST: 1
Frame 59	11.2	83.4%	67.8C	HEDEFER	DUSMAN	1 - DOST: 1
Frame 60	11.2	83.4%	67.8C	HEDEFER	DUSMAN	2 - DOST: 1
Frame 61	11.2	83.4%	67.8C	HEDEFER	DUSMAN	2 - DOST: 1
Frame 62	11.2	83.4%	67.8C	HEDEFER	DUSMAN	1 - DOST: 1
Frame 63	11.1	82.7%	68.0C	HEDEFER	DUSMAN	2 - DOST: 0
Frame 64	11.1	82.7%	68.0C	HEDEFER	DUSMAN	2 - DOST: 0
Frame 65	11.1	82.7%	68.0C	HEDEFER	DUSMAN	2 - DOST: 0
Frame 66	11.1	82.7%	68.0C	HEDEFER	DUSMAN	1 - DOST: 1
Frame 67	11.1	82.7%	68.0C	HEDEFER	DUSMAN	1 - DOST: 1
Frame 68	11.1	82.7%	68.0C	HEDEFER	DUSMAN	1 - DOST: 1
Frame 69	11.2	83.3%	70.5C	HEDEFER	DUSMAN	1 - DOST: 1
Frame 70	11.2	83.3%	70.5C	HEDEFER	DUSMAN	1 - DOST: 1
Frame 71	11.2	83.3%	70.5C	HEDEFER	DUSMAN	1 - DOST: 1
Frame 72	11.2	83.3%	70.5C	HEDEFER	DUSMAN	1 - DOST: 1
Frame 73	11.2	83.3%	70.5C	HEDEFER	DUSMAN	1 - DOST: 1
Frame 74	11.2	83.3%	70.5C	HEDEFER	DUSMAN	1 - DOST: 1
Frame 75	11.3	83.2%	71.0C	HEDEFER	DUSMAN	1 - DOST: 1
Frame 76	11.3	83.2%	71.0C	HEDEFER	DUSMAN	1 - DOST: 1
Frame 77	11.3	83.2%	71.0C	HEDEFER	DUSMAN	1 - DOST: 1
Frame 78	11.3	83.2%	71.0C	HEDEFER	DUSMAN	1 - DOST: 1
Frame 79	11.3	83.2%	71.0C	HEDEFER	DUSMAN	1 - DOST: 1
Frame 80	11.3	84.1%	71.6C	HEDEFER	DUSMAN	1 - DOST: 2
Frame 81	11.3	84.1%	71.6C	HEDEFER	DUSMAN	1 - DOST: 2
Frame 82	11.3	84.1%	71.6C	HEDEFER	DUSMAN	1 - DOST: 2
Frame 83	11.3	84.1%	71.6C	HEDEFER	DUSMAN	1 - DOST: 2
Frame 84	11.3	84.1%	71.6C	HEDEFER	DUSMAN	1 - DOST: 4
Frame 85	11.3	84.1%	71.6C	HEDEFER	DUSMAN	1 - DOST: 3
Frame 86	11.3	84.1%	71.6C	HEDEFER	DUSMAN	1 - DOST: 3
Frame 87	10.8	81.2%	71.6C	HEDEFER	DUSMAN	1 - DOST: 3
Frame 88	10.8	81.2%	71.6C	HEDEFER	DUSMAN	1 - DOST: 3
Frame 89	10.8	81.2%	71.6C	HEDEFER	DUSMAN	1 - DOST: 3
Frame 90	10.8	81.2%	71.6C	HEDEFER	DUSMAN	1 - DOST: 3
Frame 91	10.8	81.2%	71.6C	HEDEFER	DUSMAN	1 - DOST: 3
Frame 92	10.8	81.2%	71.6C	HEDEFER	DUSMAN	1 - DOST: 3
Frame 93	10.8	81.2%	72.2C	HEDEFER	DUSMAN	2 - DOST: 2
Frame 94	10.8	82.1%	72.2C	HEDEFER	DUSMAN	2 - DOST: 2
Frame 95	10.8	82.1%	72.2C	HEDEFER	DUSMAN	1 - DOST: 2
Frame 96	10.8	82.1%	72.2C	HEDEFER	DUSMAN	1 - DOST: 1
Frame 97	10.8	82.1%	72.2C	HEDEFER	DUSMAN	1 - DOST: 0
Frame 98	10.8	82.1%	72.2C	HEDEFER	DUSMAN	2 - DOST: 1

Şekil 4.16: Düşük Çözünürlüklü Kaynağın Performansının Terminal Çıktısı

Bu kapsamda, NCNN formatına dönüştürülmüş 640 giriş boyutuna sahip model kullanılarak, farklı çözünürlüklere sahip iki ayrı video kaynağı üzerinde performans karşılaştırması gerçekleştirilmiştir. Testlerde elde edilen sonuçlar Şekil 4.17’de sunulmaktadır.

Frame	FPS	CPU	Temp	HEDEFER	DUSMAN	DOST
Frame 6	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 7	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 8	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 9	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 10	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 11	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 12	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 13	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 14	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 15	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 16	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 17	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 18	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 19	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 20	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 21	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 22	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 23	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 24	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 25	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 26	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 27	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 28	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 29	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 30	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 31	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 32	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 33	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 34	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 35	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 36	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 37	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 38	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 39	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 40	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 41	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 42	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 43	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0
Frame 44	4.6	54.0%	62.2C	HEDEFER	DUSMAN	0 - DOST: 0

Şekil 4.17: Yüksek Çözünürlüklü Kaynağın Performansının Terminal Çıktısı

Düşük çözünürlüklü video kaynağı (596×336) kullanıldığında, görüntü okuma ve ölçeklendirme maliyetinin azalması sayesinde sistemin kare hızı yaklaşık 11.3 FPS seviyesine ulaşmıştır. Bu durumda işlemcinin büyük ölçüde model çıkarımına (inference) odaklanabildiği gözlemlenmiş, CPU kullanımının %81–84 aralığında gerçekleştiği tespit edilmiştir. Bu durum, sistemin mevcut donanım üzerinde yüksek verimlilikle çalıştığını göstermektedir.

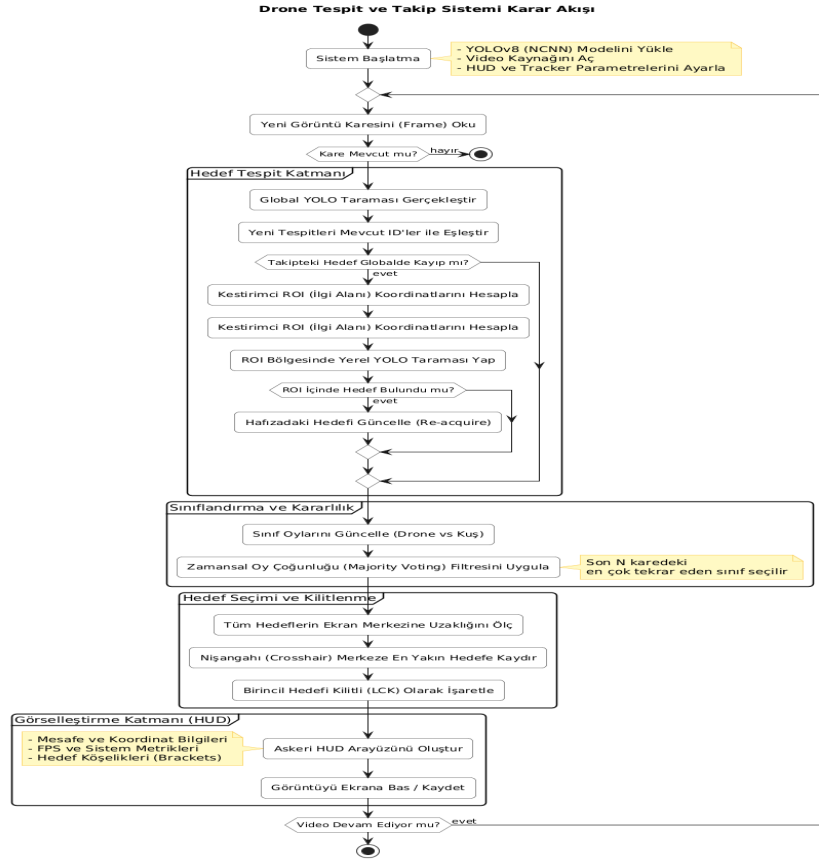
Buna karşılık, **yüksek çözünürlüklü** (4K – 3840×2160) video kaynağı kullanıldığında, özellikle video kod çözme (decoding) ve yeniden ölçeklendirme işlemlerinin oluşturduğu ek yük nedeniyle sistem performansında belirgin bir düşüş gözlemlenmiştir. Bu senaryoda kare hızı yaklaşık 4.5 FPS seviyesinde kalmış ve sistemin gerçek zamanlı çalışma kapasitesinin önemli ölçüde sınırlandığı görülmüştür.

Elde edilen sonuçlar, gömülü sistemlerde performans optimizasyonunun yalnızca model düzeyinde değil, aynı zamanda giriş veri kaynağı düzeyinde de ele alınması gerektiğini açıkça ortaya koymaktadır. Bu doğrultuda, Raspberry Pi 5 platformunda en yüksek akıcılık ve kararlılığı sağlamak amacıyla, aşırı yüksek çözünürlük yerine dengeli bir giriş çözünürlüğünün tercih edilmesinin daha uygun olduğu sonucuna varılmıştır.

4.5 Nihai Takip Sisteminin Entegrasyonu ve Performans Değerlendirmesi

Tezin önceki bölümlerinde detaylandırılan YOLOv8n tabanlı nesne tespit modeli, NCNN tabanlı donanım optimizasyonu ve geliştirilen ileri seviye takip algoritmaları, bu aşamada **yolo_roi_hud.py** ana kontrol yazılımı altında birleştirilerek tam entegre bir sistem haline getirilmiştir.

Bu entegrasyon süreci sonucunda sistem, yalnızca bir nesne tespit mekanizması olmaktan çıkarak; hedefi algılayan, zamansal kararlılık mekanizmaları ile sınıflandırma hatalarını minimize eden ve hedef önceliklendirme algoritmaları ile otonom kilitleme gerçekleştiren bütünlük bir takip mimarisine dönüştürülmüştür. Geliştirilen sistemin çalışma prensibi ve karar alma süreçleri, Şekil 4.18’de sunulan akış diyagramı üzerinden detaylı olarak gösterilmektedir.



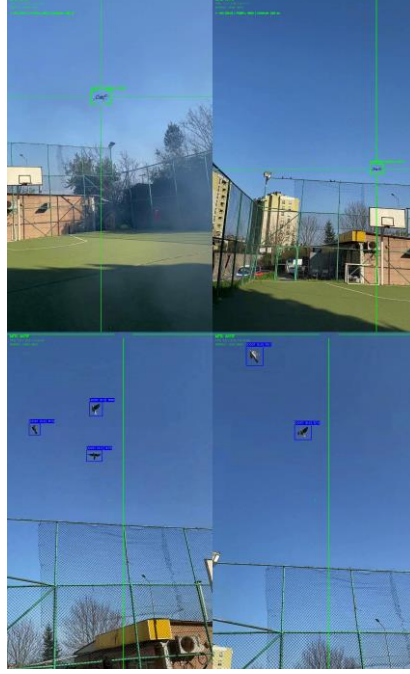
Şekil 4.18: Sistemin Akış Diyagramı

Geliştirilen nihai sistem mimarisi, Raspberry Pi 5 gömülü platformu üzerinde gerçek dünya koşullarını temsil eden test videoları kullanılarak kapsamlı bir doğrulamaya tabi tutulmuştur. Test senaryoları; farklı mesafe, ölçek ve hedef sayısını içerecek şekilde çeşitlendirilmiş olup sistemin hem tespit hem de takip performansı değerlendirilmiştir.

Elde edilen sonuçlar incelendiğinde, sistemin global tarama aşamasında hedefleri başarıyla tespit edebildiği, tespit kaybı yaşanan durumlarda ise ROI tabanlı yeniden tespit (re-acquisition) mekanizması ile hedefi kısa sürede tekrar yakalayabildiği gözlemlenmiştir. Ayrıca, çoklu hedef senaryolarında sistemin aktif hedef önceliklendirme algoritması sayesinde nişangahı en yüksek önceliğe sahip hedef üzerinde kararlı biçimde tutabildiği tespit edilmiştir.

Şekil 4.19’da sunulan örnek görüntülerde, sistemin farklı senaryolar altında hedefleri eş zamanlı olarak tespit ettiği, kimlik ataması yaptığı ve seçili hedef üzerinde stabil

bir kilitleme gerçekleştirdiği görülmektedir. Bu bulgular, önerilen mimarinin gerçek zamanlı ve gömülü sistem koşullarında etkin bir şekilde çalışabildiğini ortaya koymaktadır.



Şekil 4.19: Test Sonuçları

Şekil 4.19’de sunulan test sonuçları incelendiğinde, YOLO tabanlı nesne tespiti, ROI yönetimi, takip algoritmaları ve görsel HUD arayüzünün eş zamanlı olarak çalışmasına rağmen sistemin Raspberry Pi 5 platformu üzerinde ortalama 7.5 FPS kare hızına ulaştığı görülmektedir.

Elde edilen bu performans, gömülü bir platform üzerinde çok katmanlı bir takip hattının (pipeline) gerçek zamanlıya yakın koşullarda etkin bir şekilde çalıştırılabildiğini göstermektedir. 7.5 FPS seviyesindeki kararlı işlem hızı, hedefin izlenmesi ve otonom nişangahın kilitlenmesi açısından yeterli bir akıcılık sunarak sistemin operasyonel gereksinimlerini karşılamaktadır.

Bu bulgular doğrultusunda, önerilen mimarinin hem doğruluk hem de performans açısından gömülü sistemler üzerinde uygulanabilir olduğu doğrulanmıştır. Bu kapsamda, tezin üçüncü bölümü olan “Deney ve Yöntem” aşaması tamamlanmıştır.

5. PAN-TİLT DESTEKLİ GERÇEK ZAMANLI OTONOM TAKİP MEKANİZMASININ UYGULANMASI

Çalışmanın 4.5 numaralı bölümünde ayrıntılı olarak açıklanan **yolo_roi_hud.py** scripti kapsamında; YOLO tabanlı dost-düşman sınıflandırması, ROI yani büyüteç bölgesi üzerinden yeniden tespit algoritması ve zamansal oy çoğunluğu gibi yazılımsal yöntemler, önceden kaydedilmiş video verileri üzerinde test edilerek başarıyla doğrulanmıştır. Bu aşamada sistemin dijital ortamda hedefleri algılama, sınıflandırma ve işaretleme görevlerini başarılı şekilde yerine getirdiği görülmüştür. Ancak gerçek zamanlı ve otonom bir hava savunma prototipinin temel amacı, hedefleri yalnızca ekran üzerinde belirlemek değil; kameranın fiziksel görüş açısını hedefe yönlendirerek sürekli bir takip mekanizması oluşturmaktır.

Bu amaç doğrultusunda, video tabanlı senaryolarda etkinliği kanıtlanan otonom takip yapısı, temel algoritmik mimarisi korunarak **main2.py** isimli nihai ana kontrol scriptine aktarılmıştır. Yeni sistemde, statik video dosyaları yerine iki eksenli Pan-Tilt mekanizması üzerine yerleştirilen Raspberry Pi Camera 3 Wide modülünden alınan canlı görüntü akışı giriş verisi olarak kullanılmıştır. Böylece sistem, gerçek zamanlı görüntü işleme ve fiziksel yönlendirme kabiliyetine sahip hale getirilmiştir. Sistemde yapay zekâ algoritmalarını çalıştıran Raspberry Pi 5 ile servo motorlar arasındaki haberleşme, PCA9685 servo motor sürücüsü ve I2C protokolü üzerinden sağlanmıştır. Raspberry Pi 5, canlı kamera görüntüsünde “Düşman” olarak sınıflandırılan hedefin ekran üzerindeki konumunu main2.py içerisinde analiz eder. Hedef merkezinin görüntü merkezine göre yatay ve dikey eksenlerdeki sapma miktarı hata değeri olarak hesaplanır. Bu hata bilgisi PCA9685 sürücüsüne iletilerek servo motorların pan ve tilt eksenlerinde gerekli açısal düzeltmeleri yapması sağlanır. Böylece yazılım tarafından üretilen takip kararları, fiziksel sistem üzerinde anlık motor hareketlerine dönüştürülür.

Gerçekleştirilen bu donanım-yazılım entegrasyonu, sistemi yalnızca görüntü işleme temelli bir yapı olmaktan çıkarak çevresiyle fiziksel olarak etkileşime girebilen otonom bir mekatronik prototipe dönüştürmüştür. Canlı görüntüden elde edilen verilerin işlenmesi, hedef konum hatasının hesaplanması ve servo motorlar

aracılığıyla kameranın hedefe yönlendirilmesi sayesinde sistem kapalı çevrim çalışan, gerçek zamanlı hedef takibi yapabilen bir savunma sistemi modeli haline gelmiştir.

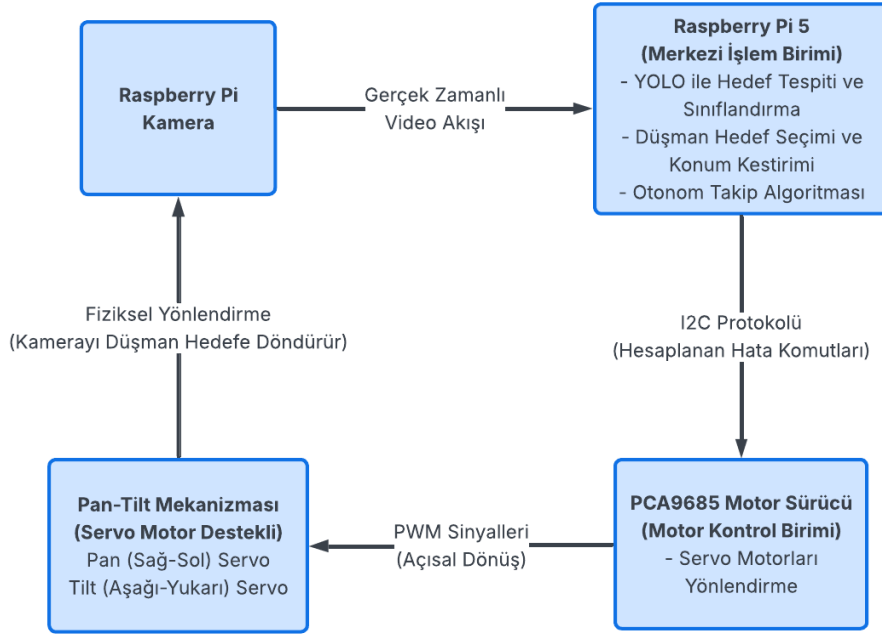
5.1 Otonom Takip Sisteminin Donanım Mimarisi ve İletişim Protokolleri

Gerçek zamanlı hedef takip sisteminin donanım yapısı; Raspberry Pi 5, PCA9685 servo motor sürücü modülü ve iki eksenli Pan-Tilt servo mekanizması temel alınarak oluşturulmuştur.

Raspberry Pi 5 gibi işletim sistemi üzerinde çalışan kartlarda, GPIO pinlerinden doğrudan PWM sinyali üretmek her zaman kararlı sonuç vermeyebilir. İşlemci üzerinde oluşan anlık yük değişimleri, servo motorlarda titreme ve konum kararsızlığına neden olabilmektedir. Bu durumu önlemek ve motor hareketlerini daha stabil hale getirmek amacıyla Raspberry Pi 5 ile servo motorlar arasına PCA9685 sürücü modülü eklenmiştir. Raspberry Pi 5 ile PCA9685 arasındaki haberleşme ise I2C protokolü üzerinden gerçekleştirilmiştir.

Raspberry Pi 5 tarafından hesaplanan hedef sapma bilgileri, I2C hattı aracılığıyla PCA9685 modülüne aktarılır. PCA9685, üzerinde bulunan bağımsız clock yapısı sayesinde işlemci yükünden etkilenmeden 12-bit çözünürlükte kararlı PWM sinyalleri üretir. Bu sinyaller Pan ve Tilt eksenlerinde görev yapan servo motorlara iletilerek kameranın hedefe daha düzgün, sarsıntısız ve kontrollü şekilde yönelmesi sağlanır.

Bu donanım iletişim yapısı, sistemin gerçek zamanlı takip performansını artırmakta ve otonom hedef takibinin daha kararlı şekilde gerçekleştirilmesine katkı sağlamaktadır. Pan-tilt destekli otonom hedef takip sisteminin temel donanım bileşenleri ile bu bileşenler arasındaki iletişim ve kontrol ilişkileri Şekil 5.1’de sunulmuştur.



Şekil 5.1: Pan-Tilt Destekli Otonom Hedef Takip Sisteminin Donanım Mimarisi

5.2 Pan-Tilt Destekli Otonom Takip Algoritmasının Tasarımı

Sistemin otonom hareket kabiliyetini sağlayan temel mekanizma, yazılım (görüntü işleme) ile donanım (servo motorlar) arasında kurulan sürekli bir geri besleme (feedback) döngüsüdür. Geliştirilen yazılım mimarisi içerisinde yer alan PanTiltController sınıfı, bu kapalı çevrim kontrol döngüsünün merkezinde yer almaktadır. Algoritmanın temel işlevi; HUD arayüzü üzerinde kilitlenilen (LCK) düşman hedefin anlık koordinatlarını (X_{hedef} , Y_{hedef}), kameranın görüş alanının tam merkezine (X_{merkez} , Y_{merkez}) getirmek ve otonom takip boyunca o noktada tutmaktır.

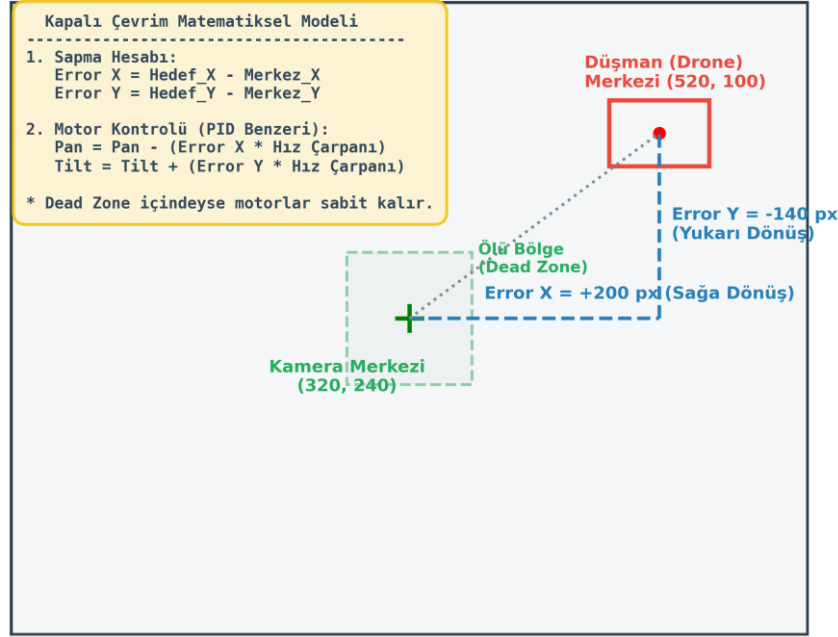
Bu otonom merkezleme süreci, hedef ile ekran merkezi arasındaki piksel sapmasının (Error) anlık olarak hesaplanmasına dayanmaktadır:

- Yatay Sapma (Error_X): $X_{\text{hedef}} - X_{\text{Merkez}}$
- Dikey Sapma (Error_Y): $Y_{\text{hedef}} - Y_{\text{Merkez}}$

Hesaplanan hata değerleriyle hedefin **kamera merkezine** çekilmesini sağlayan matematiksel model Şekil 5.2'de sunulmuştur. Şemada belirtilen "Ölü Bölge (Dead Zone)" eşiği, küçük piksel oynamalarının motorlarda titremeye (jitter) yol açmasını

engellemek için algoritmaya eklenmiştir. Hedefin merkezden sapma miktarı bu bölge sınırları içerisindeyse motorlar sabit kalır ve yalnızca görsel takip yapılır. Pan-tilt kontrol algoritmasında kullanılan yatay ve dikey hata hesaplama yapısı ile ölü bölge sınırlarının çalışma prensibi Şekil 5.2’de sunulmuştur

Pan-Tilt Otonom Takip Algoritması: Sapma (Error) Hesaplama Mimarisi



Şekil 5.2 : Sapma (Error) Hesaplama Mimarisi

Şekildeki matematiksel modele göre; hedef sapma miktarının ölü bölgeyi aşması durumunda aktif motor kontrol mekanizması devreye girer. Hesaplanmış olan hata (error) verileri, sistemin dinamiklerine uygun olarak belirlenen bir takip hızı çarpanı ile çarpılarak yeni Pan (Yatay) ve Tilt (Dikey) hedef açıları elde edilir. Elde edilen bu yeni açılar, Pan-Tilt mekanizmasının fiziksel sınırlarına zarar vermemek adına donanımsal olarak 0° ile 180° arasında sınırlandırılır (clamping).

Sınırlandırılan bu nihai açı komutları, I2C iletişim veri yolu üzerinden motor sürücüsüne aktarılır. Servo motorların kamerayı yeni açılara döndürmesiyle birlikte hedef tekrar kamera merkezine yaklaşır. Sistem, manevra sonrasında yeni bir görüntü karesi alarak sapma hatasını yeniden hesaplar. Sistem çalıştığı sürece saniyede ortalama 10 defa (~10 FPS) tekrarlanan bu sürekli kapalı çevrim (closed-loop)

döngüsü, sarsılmaz ve tam otonom bir donanımsal takip sistemini meydana getirmektedir. Şekil 5.3'de sunulmuştur



Şekil 5.3: Gerçek Zamanlı Arayüzdeki Hesaplanan Bilgiler

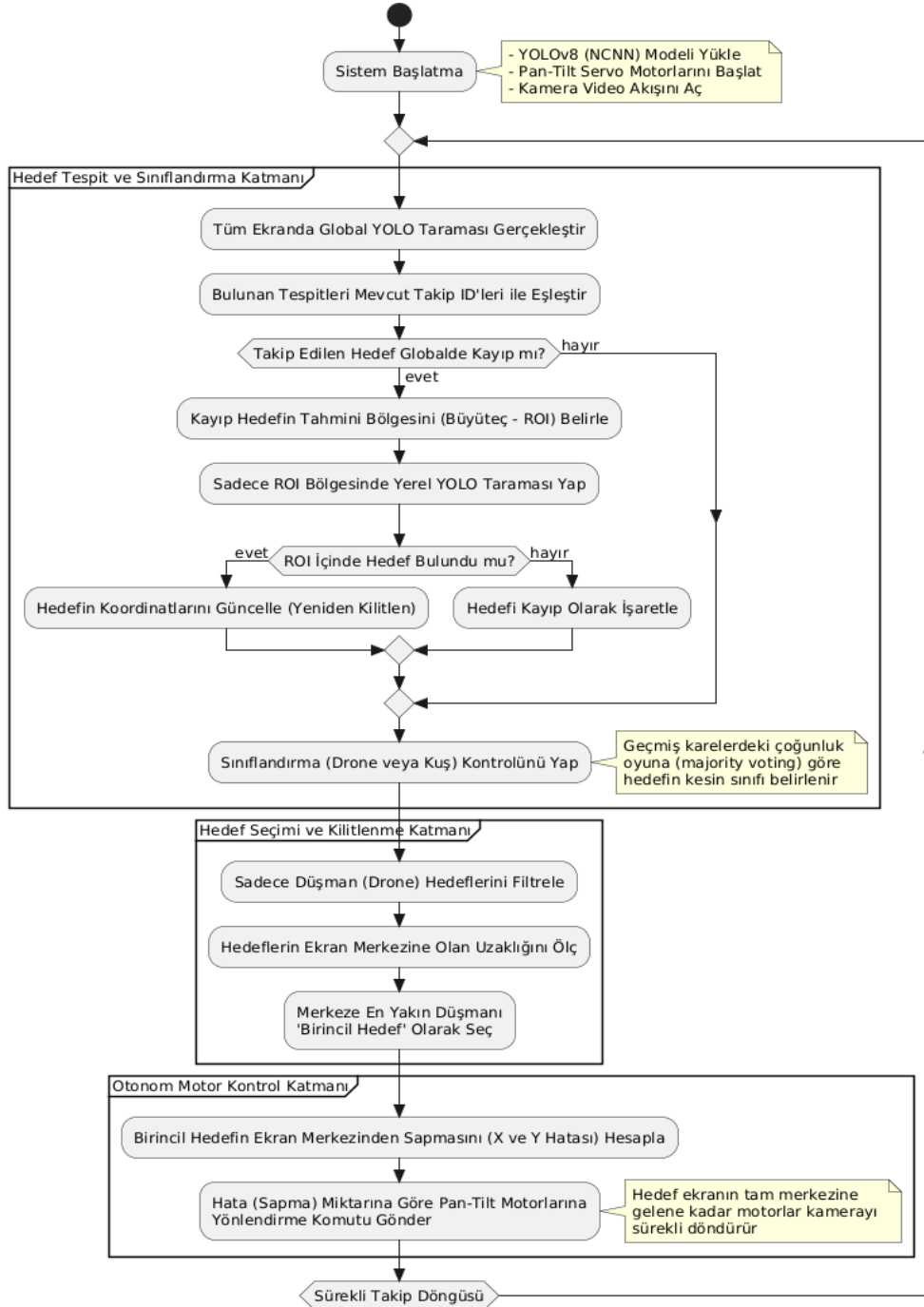
5.3 Nihai Sonuç Mimarisi

Tez kapsamında geliştirilen otonom hava savunma sisteminin teorik tasarımları, yazılım mimarisi ve donanım algoritmaları başarıyla fiziksel bir prototipe dönüştürülmüştür. Sistemin nihai fiziksel kurulumu, harici bir bilgisayara ihtiyaç duymadan arazi şartlarında (Edge) bağımsız olarak çalışabilir bir yapıda kurgulanmıştır.

Sistemin mimari bütünlüğü üç ana katmandan oluşmaktadır: Veri toplama katmanı (Kamera), Merkezi işleme ve karar katmanı (Raspberry Pi 5) ve Eyleyici katmanı (PCA9685 ve Servo Motorlar). Bu üç katman, bulut (cloud) sunucusuna ihtiyaç duymadan tamamen uç cihaz (Edge AI) prensibiyle kapalı çevrim (closed-loop) olarak haberleşmektedir.

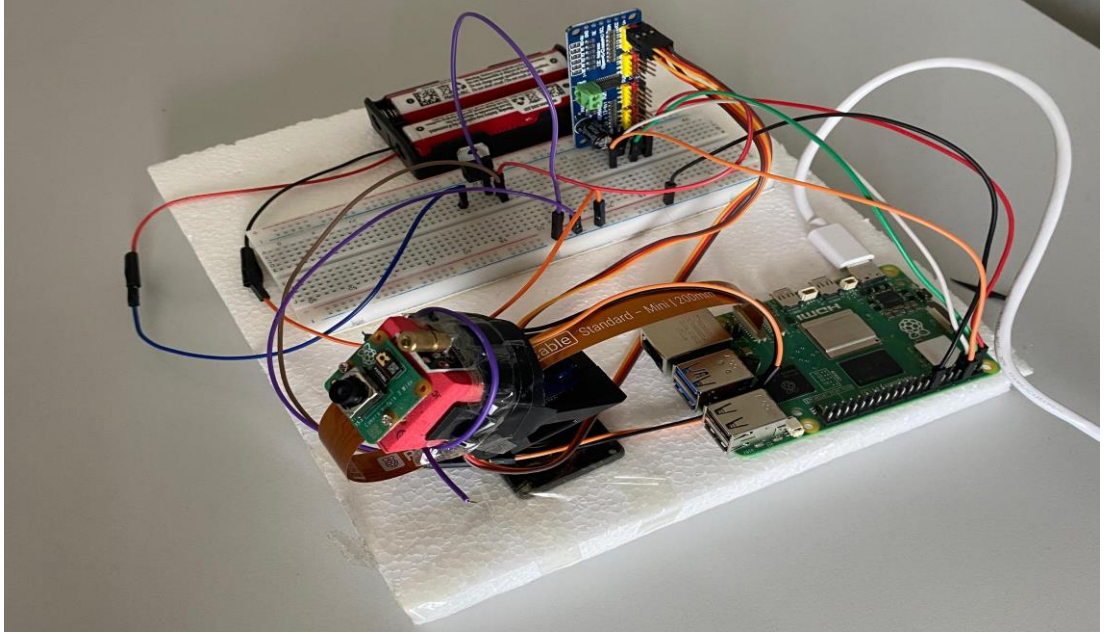
Şekil 5.4'te görüldüğü üzere sistem ilk olarak kamera görüntüsü üzerinde hedef tespiti ve sınıflandırma işlemini gerçekleştirmektedir. Tespit edilen hedefler takip kimlikleriyle eşleştirilmekte, geçici hedef kayıplarında önceki karelerden elde edilen bilgiler kullanılarak hedef konumu korunmaktadır. Düşman olarak sınıflandırılan hedefler arasından ekran merkezine en yakın olan hedef seçilmekte ve yatay-dikey

sapma değerleri hesaplanmaktadır. Elde edilen hata değerlerine göre pan-tilt motorlarına yönlendirme komutları gönderilerek hedefin görüntü merkezinde tutulması sağlanmaktadır.



Şekil 5.4: Nihai Sistem Mimarisi

Sistemin gerek dnyadaki fiziksel entegrasyonu ise ekil 5.5'de sunulmuştur.



ekil 5.5: Projenin Fiziksel Hali

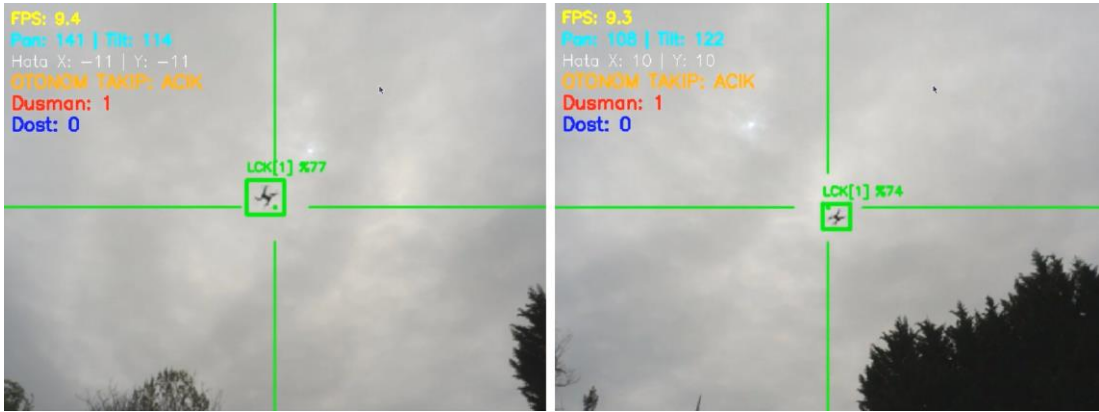
Sistemde yer alan tm donanım bileşenleri; Raspberry Pi 5, motor src modl ve g kaynaėı, kararlı bir alıřma ortamı oluřturacak řekilde ortak bir platform zerinde bir araya getirilmiřtir. İki eksenli hareket yeteneėine sahip Pan-Tilt mekanizması ise hedeflerin daha geniř bir grř aısıyla izlenebilmesi amacıyla sistemin st kısmına konumlandırılmıřtır. Sistemin enerji gereksinimi tařınabilir bataryalar aracılıėıyla karřılanarak saha testlerinde ihtiya duyulan hareket kabiliyeti ve donanımsal baėımsızlık saėlanmıřtır. Oluřturulan bu nihai mimari, projenin yalnızca yazılım tabanlı bir simlasyondan ibaret olmadıėını; gerek dnya problemlerine otonom ve fiziksel zmler sunabilen mekatronik bir savunma prototipi niteliėi tařıdıėını gstermektedir.

5.4 Aık Alan Saha Testleri ve Deneysel Sonular

Sistemin fiziksel nihai mimarisi tamamlandıktan sonra, geliřtirilen otonom hava savunma prototipi laboratuvar ortamından ıkarılarak gerek dnya kořullarında aık alan saha testlerine tabi tutulmuřtur. Bu testlerin temel amacı; gneř ıřıėı, deėiřen

arka planlar (gökyüzü, ağaçlar, binalar) ve rüzgar gibi zorlu çevresel etkenler altında sistemin hedef tespit doğruluğunu ve kapalı çevrim (closed-loop) takip kararlılığını ölçmektir.

Saha testleri, dış ortamda çeşitli irtifa ve hızlarda hareket eden gerçek bir drone hedefine karşı gerçekleştirilmiştir. Şekil 5.6'da görüldüğü üzere, sistem başlatılıp kamera canlı akışa geçtiğinde, YOLOv8n (NCNN) modeli değişen aydınlatma koşullarına rağmen drone hedefini yüksek doğrulukla tespit etmiş ve otonom takip mekanizması hedefi vizörde merkeze alarak başarıyla kilitlenmiştir. Görüntü üzerine yansıtılan taktiksel HUD arayüzünde de açıkça izlenebildiği gibi; sistem hedefi merkezde tutabilmek için yatay (Pan) ve dikey (Tilt) motor açılarını anlık olarak hesaplamakta ve donanımı doğru koordinatlara yönlendirmektedir.



Şekil 5.6: Gerçek Zamanlı Test Sonuçları

Testlerin devamında hedefin ani manevralar yaptığı ve yükseklik değiştirdiği farklı zorluk senaryoları da incelenmiştir. Şekil 5.7'de sunulan test kesitinde, hedefin ekran merkezinden sapmaya çalıştığı anlarda sistemin ürettiği X-Y eksenli sapma hataları (Error) ve buna karşılık motorlara gönderilen otonom düzeltme açıları net bir şekilde görülmektedir. Sistem; dış ortamda derin öğrenme tabanlı görüntü işleme, dinamik hata hesaplama ve I2C üzerinden sürekli motor sürme işlemlerinin tamamını eş zamanlı olarak yürütmesine rağmen, HUD ekranındaki verilerin de doğruladığı üzere ortalama **8.0 - 10.5 FPS** aralığında çalışarak son derece stabil bir performans sergilemiştir.



Şekil 5.7 : Gerçek Zamanlı Test Görüntüsü

Gerçekleştirilen bu test senaryoları boyunca sistemin saha performansı şu üç ana başlık altında deneysel olarak doğrulanmıştır:

1. **Görsel Tespit Direnci:** Hedefin uzaklaşıp küçüldüğü anlarda ROI (Büyüteç) algoritması gecikmesiz olarak devreye girerek hedef kaybını engellemiş, Zamansal Oy Çoğunluğu (Temporal Majority Voting) algoritması ise dış ortamdaki görsel kararsızlıkları tamamen filtrelemiştir.
2. **Düşük Gecikmeli Motor Kontrolü:** PCA9685 sürücüsü üzerinden gönderilen PWM sinyalleri, yazılımın ürettiği sapma hatalarını neredeyse sıfır gecikmeyle (low-latency) fiziksel manevralara çevirerek, hedefin vizörden dışarı kaçmasına izin vermemiştir.
3. **Mekanik Kararlılık:** Geliştirilen Ölü Bölge (Dead Zone) algoritması, rüzgar veya çok küçük piksel oynamaları kaynaklı oluşabilecek motor titremelerinin (jitter) önüne geçerek pürüzsüz bir donanımsal takip süreci sağlamıştır.

Elde edilen bu deneysel sonuçlar ve sahada kaydedilen gerçek zamanlı takip görüntüleri, projenin sadece laboratuvar ortamında değil, dış dünya koşullarında da çalışan tam otonom bir hava savunma prototipi olarak başarıyla hayata geçirildiğini kanıtlamaktadır

SONUÇLAR VE ÖNERİLER

Bu tez çalışmasında, hava sahası güvenliğini tehdit eden insansız hava araçlarının (İHA) gömülü platformlar üzerinde gerçek zamanlı olarak tespit edilmesini ve otonom takip edilmesini sağlayan, yapay zeka destekli donanım-yazılım bütünleşik bir hava savunma prototipi başarıyla geliştirilmiştir. Laboratuvar ortamında ve açık alan saha testlerinde gerçekleştirilen deneysel çalışmalar neticesinde elde edilen temel bulgular şu şekildedir:

- **Model Optimizasyonu ve Gömülü Sistem Başarımı:** Derin öğrenme tabanlı YOLOv8n nesne tespiti modeli, Raspberry Pi 5 platformu üzerinde doğrudan çalıştırılmak yerine NCNN altyapısına dönüştürülmüştür. Bu sayede işlemcinin donanım kaynakları maksimum verimle kullanılmış ve standart PyTorch modellerine kıyasla çıkarım hızında kayda değer bir artış sağlanmıştır.
- **Yazılımsal Kararlılık (Algoritmik Mukavemet):** Geleneksel izleme metodlarının gömülü sistem kısıtları altındaki zayıflıkları saptanmış; geliştirilen "Vision-Only" (Sadece Görüntü Tabanlı) ROI mantığı ve "Zamansal Oy Çoğunluğu" (Temporal Majority Voting) algoritmaları sayesinde hedef sınıflandırmasındaki kararsızlıklar giderilmiş, "Kuş" ve "Drone" sınıfları arasındaki ayrım başarıyla yapılmıştır.
- **Kapalı Çevrim Elektromekanik Takip:** Görüntü işleme katmanından elde edilen hedef merkez koordinatları ile ekran merkezi arasındaki sapma (error) değerleri, I2C protokolü üzerinden PCA9685 servo motor sürücüsüne iletilmiştir. Bu sayede yazılımsal kararlar, iki eksenli (Pan-Tilt) donanımsal harekete dönüştürülerek tam otonom bir kapalı çevrim kontrol mekanizması kurulmuştur.
- **Saha Performansı:** Dış ortamda hareketli hedeflere karşı gerçekleştirilen gerçek zamanlı testlerde; değişen ışık, rüzgar ve arka plan koşullarına rağmen sistemin hedefe başarıyla kilitlendiği gözlemlenmiştir. Tüm algoritmik ve mekanik işlemlerin eş zamanlı yürütülmesine rağmen sistemin **8.0 - 10.5 FPS** aralığında stabil bir operasyonel hız sergilediği kanıtlanmıştır.

Öneriler (Gelecek Çalışmalar)

Geliştirilen bu prototipin ilerleyen çalışmalarda daha profesyonel ve ticari standartlara ulaştırılabilmesi için aşağıdaki geliştirmelerin yapılması önerilmektedir:

- **Gece ve Zorlu Hava Koşulları Entegrasyonu:** Sisteme mevcut RGB (görünür bölge) kameralarına ek olarak termal veya kızılötesi (IR) kamera modüllerinin entegre edilmesi, hedeflerin gece şartlarında ve sisli hava koşullarında da ısı izinden tespit edilebilmesine olanak tanıyacaktır.
- **Donanımsal Yapay Zeka Hızlandırıcısı (NPU) Kullanımı:** Gelecek versiyonlarda Raspberry Pi 5 işlemcisi yerine, doğrudan tensör işlemleri ve yapay zeka çıkarımı için tasarlanmış donanımsal hızlandırıcılara sahip (NVIDIA Jetson Orin Nano vb.) platformlara geçiş yapılması, sistemin kare hızını (FPS) 30 ve üzerine çıkararak donanımsal tepki süresini ciddi oranda iyileştirecektir.
- **Çoklu Sensör Füzyonu:** Kamera tabanlı görüş alanının dışında (kör noktada) kalan hedeflerin erken tespiti için sisteme yönlü kompakt radar sensörleri eklenebilir. Radar-Kamera füzyonu sayesinde, İHA henüz görüş alanına girmeden koordinatları kestirilip kamera doğrudan o bölgeye yönlendirilebilir.

KAYNAKLAR

- [1] Ultralytics YOLOv8 Documentation. (2023). "YOLOv8: Real-Time Object Detection and Tracking Solutions." Eriřim adresi: <https://docs.ultralytics.com/>
- [2] Jocher, G., Chaurasia, A., & Qiu, J. (2023). "Ultralytics YOLOv8 (Version 8.0.0)." Software available from <https://github.com/ultralytics/ultralytics>
- [3] Raspberry Pi Foundation. (2023). "Raspberry Pi 5 Technical Specifications and Documentation." Eriřim adresi: <https://www.raspberrypi.com/documentation/>
- [4] Redmon, J., et al. (2016). "You Only Look Once: Unified, Real-Time Object Detection." CVPR.
- [5] Tencent NCNN. (2024). "High-performance neural network inference framework optimized for the mobile platform." GitHub Wiki.
- [6] OpenCV Documentation. (2024). "Open Source Computer Vision Library - Reference Manual." Eriřim adresi: <https://docs.opencv.org/>
- [7] Roboflow. Eriřim adresi: <https://roboflow.com/>
- [8] Kalman, R. E. (1960). "A New Approach to Linear Filtering and Prediction Problems." Journal of Basic Engineering.
- [9] Luo, W., et al. (2021). "Multiple object tracking: A review." Artificial Intelligence.
- [10] NXP Semiconductors. (2015). "PCA9685: 16-channel, 12-bit PWM Control." Data Sheet.

ÖZGEÇMİŞ

2003 yılında İstanbul’da doğdu. İlk, orta ve lise eğitimini İstanbul’da tamamladı. 2021 yılında Kocaeli Üniversitesi Mühendislik Fakültesi Elektronik ve Haberleşme Mühendisliği Bölümünde lisans eğitimine başladı. 2026 yılında Elektronik ve Haberleşme Mühendisliği Bölümünden mezun olma durumu bulunmakta.